UNIT-IV

Search and Analytics Middleware

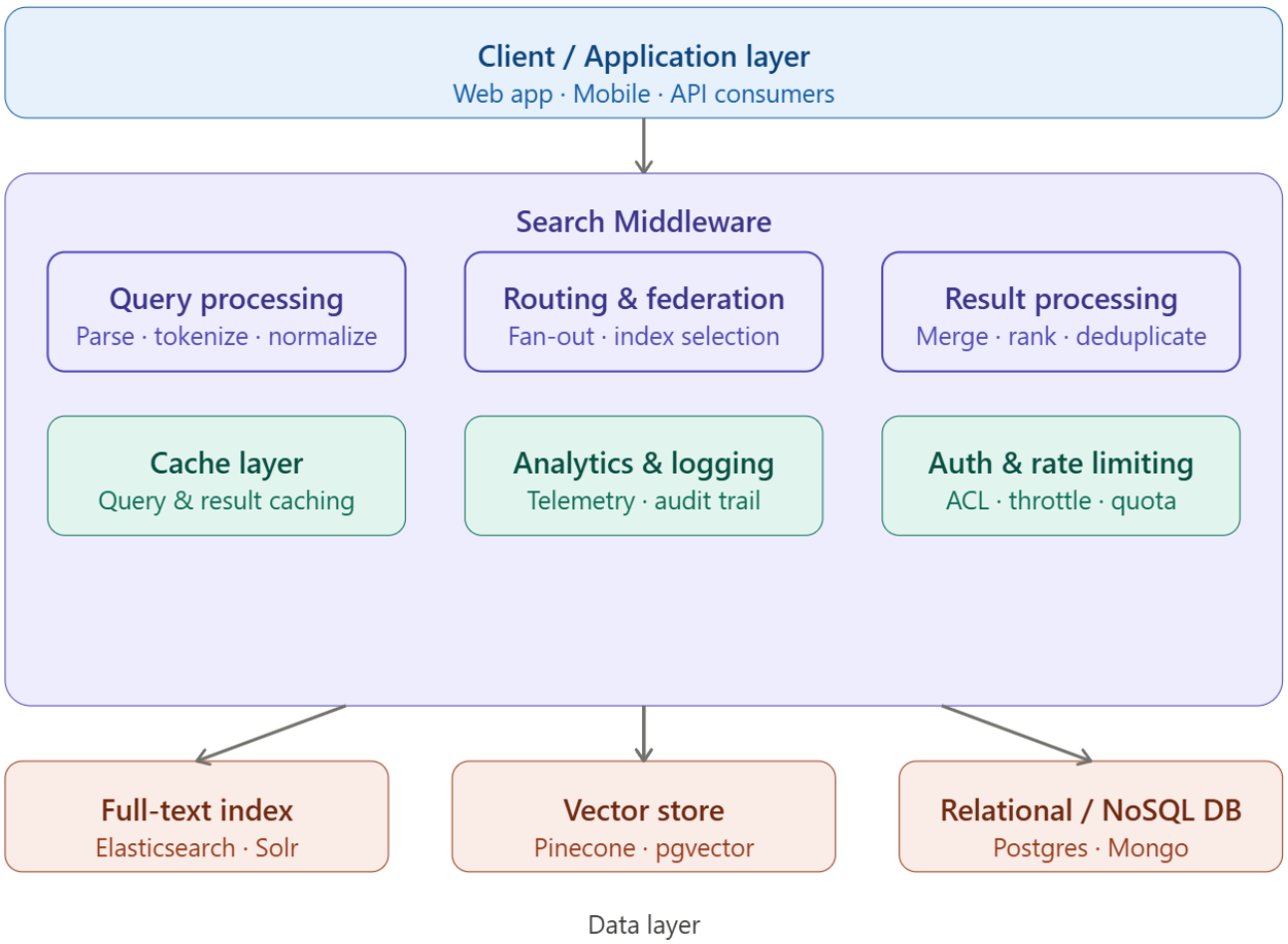
Fundamentals of Search Middleware, Need for Search and Analytics, Elasticsearch Architecture: Nodes and Clusters, Index, Document, and Shards, Inverted Index Concept, Indexing and Querying Mechanisms: Document indexing, Mapping, Basic Query DSL for Searching, Full-Text Search Capabilities, Introduction to aggregations, Basic Aggregation Types: Metrics and Bucketing Aggregations, Elasticsearch Use Cases: Log and Event Analytics, Search Applications, Monitoring and Observability.

Fundamentals of Search Middleware

Search middleware sits between your application layer and your underlying data stores. Search Middleware is a specialized software layer designed to facilitate efficient data retrieval, full-text search, and complex querying across large-scale distributed datasets. Unlike traditional relational databases optimized for structured transactions, search middleware is architected for unstructured or semi-structured data and high-performance relevance-based retrieval.

For Example:

Traditional SQL databases answer exact-match queries extremely well (SELECT * FROM orders WHERE id = 42). However, they struggle with questions like: **"Show me products similar to running shoes under 5000 INR with good reviews."** This is where search middleware excels.



Query processing pipeline — When you type a search, the system tidies it up before looking anything up. It splits your sentence into individual words (*tokenization*), simplifies words to their root form — "running" becomes "run" (*stemming*). It throws away useless words like "the" and "is" (*stop word removal*). It also expands your search — if you type "car", it also looks for "automobile" and "vehicle" (*synonym expansion*). And if you made a typo, it quietly fixes it (*spell correction*).

Routing and federation — Imagine you need to research a topic and you ask three different librarians simultaneously — one specialises in exact words, one in related concepts, one in structured records. The middleware does exactly this: it sends your search to several databases at the same time, then collects and combines all the answers. This is called *federated search*.

Ranking and scoring — Each database gives back results with its own "score" of how relevant they are — but these scores don't use the same scale, like comparing marks out of 10 vs. out of 100. The middleware converts everything to the same scale, then boosts results that are newer or that other users actually clicked on, before sorting the final list for you.

Caching strategies — If 1,000 people search for the same thing, it's wasteful to look it up 1,000 times. Instead, the system saves the answer the first time and hands it out instantly to everyone else (*caching*). The tricky part is knowing when a saved answer is out of date and needs to be refreshed (*cache invalidation*).

Authentication, authorization, and access control — Before your search even reaches the database, the system checks who you are and quietly adds a filter: "only return results this person has permission to read." So, two people searching the same word can get completely different results based on their access level — without either person seeing the filter being applied.

Observability layer — Every single search is silently recorded: how long it took, how many results came back, which database was used, and who searched. This data feeds into dashboards so engineers can spot problems and improve results. It also tracks which results users actually clicked — so the system can learn over time what "good results" looks like.

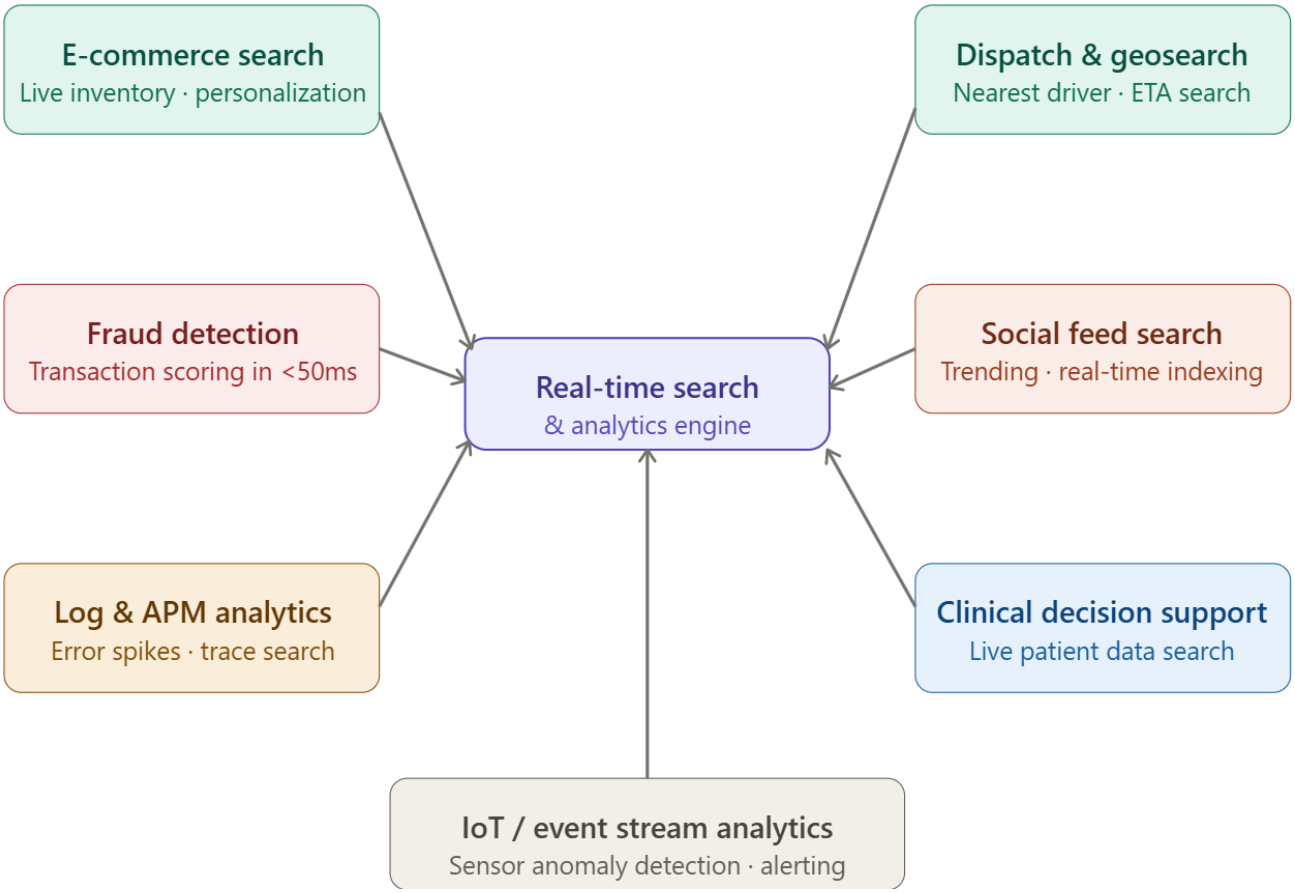
Need for Search and Analytics

Traditional batch-processing systems cannot meet modern expectations. Users expect sub-100ms responses, live inventory counts, real-time fraud signals, and dashboards that reflect the world as it is right now — not as it was an hour ago. The fundamental problem search and analytics solves is making large volumes of heterogeneous data *queryable at human speed*.

E-commerce search — "Don't show what you can't sell": When you search "red running shoes size 10", the results must reflect *live* stock. If a pair sold out 2 seconds ago, showing it still available wastes your time and ruins checkout. The system listens to stock changes as they happen and instantly updates search results. It also uses your past clicks to push the most relevant items to the top.

Fraud detection — "Catch the bad transaction in under 50ms": Every time you pay, a system quietly checks: Is this your normal behaviour? Is this the 5th payment in 10 seconds? Is this device linked to known fraud? It pulls your recent history from memory, runs all these checks, and decides to approve or flag — all before the payment app even gets a response, in under 50 milliseconds.

Log analytics — "Finding a needle in a billion haystacks, instantly": When something breaks at 3am, an engineer types something like "errors in checkout — last 15 minutes" and needs an answer in under a second — across billions of stored log lines. Tools like Elasticsearch make this possible by organising logs in a way that's fast to search. Kibana sits on top as the visual dashboard.



Ride-hailing dispatch — "Find the nearest driver in 2 seconds": When you request a ride, the system instantly searches for all available drivers within a few kilometres, filters by car type, ranks by who can arrive fastest, and locks one in — all within 2–3 seconds. It uses special location-based indexing (think of the map divided into tiny grid cells) to make this fast even across an entire city.

Social media trending — "Tweets from 2 seconds ago already searchable": When you search a hashtag on X/Twitter, you expect to see posts from moments ago — not from an hour ago. The system writes new posts to the search index within 1–2 seconds of them being created. Trending topics aren't calculated once a day — they're computed continuously using a sliding window over the live stream of posts.

Clinical decision support — "Alert the doctor before the patient deteriorates": In an ICU, a patient's vitals stream in constantly from monitors. The system watches for warning patterns in real time and alerts a doctor the moment a patient's condition score crosses a danger threshold. A doctor

can also search "all abnormal lab results this admission" and see results that include a test done 10 minutes ago.

IoT & sensor analytics — "Spot the overheating machine before it breaks": A factory with 50,000 sensors streams data every second. The system compares current readings against normal baselines and raises an alert if, say, a motor is vibrating abnormally — before it actually fails. Engineers can also search across the whole factory: "all machines with high vibration in the last hour" and get an instant answer.

What is Elasticsearch?

- **Elasticsearch** is an **open-source** and **distributed** search and **analytics** engine built based on the **Apache Lucene.**
- It is designed to handle large volumes of data and provide near **real-time** search capabilities across various types of **structured** and **unstructured** data.
- Elasticsearch is part of the Elastic Stack, which includes other tools like **Kibana** for **data visualization**, **Beats** for **data shipping,** and **Logstash** for **data processing.**
- Elasticsearch stores data in JSON format and making it easy to index and search structured and unstructured data.
- Elasticsearch supports the concept of multi-tenancy, allowing us to **index** and search data for multiple applications or users within a single **cluster**.

Comparison Between Elasticsearch and Traditional Databases

Aspect	**Elasticsearch**	**Traditional Databases**
Data Model	Document-oriented, JSON-based	Relational, table-based with predefined schemas
Querying	Uses Elasticsearch Query DSL	Uses SQL
Scalability	Horizontally scalable	Limited scalability, often requires complex strategies
Performance	Real-time search and analytics	Structured query performance may degrade with scale
Consistency	Eventual consistency	Strong consistency
Use Cases	Full-text search, real-time analytics	Transactional applications, structured data management

Elasticsearch architecture

Elasticsearch is a distributed, open-source search and analytics engine designed for handling large volumes of data. Its unique architecture allows for real-time search and analytics, making it a popular choice for applications such as logging, full-text search, and business intelligence. One of the key

features of Elasticsearch's architecture is its use of a distributed data model, which allows for horizontal scalability and high availability.

1. Distributed Nature

Elasticsearch is **inherently** distributed, meaning it can run on a cluster of **interconnected nodes** to distribute data and workload across multiple machines. This distributed architecture allows Elasticsearch to scale horizontally, enabling it to handle large amounts of data and support high query loads.

Cluster

- A cluster is a group of one or more Elasticsearch nodes that work together and share the same data.
- It is identified by a unique cluster name.
- A cluster provides distributed storage, scalability, and fault tolerance.
- Nodes in a cluster communicate with each other to coordinate indexing, searching, and data replication.

Node

- A node is a single running instance of Elasticsearch on a physical or virtual machine.
- Each node is uniquely identified by a node ID and node name.
- Nodes store data and participate in the cluster's indexing and search operations.
- Nodes communicate with other nodes in the cluster to maintain data consistency and availability.

2. Indexing and Data Model

Elasticsearch organizes and stores data in the form of documents within indices. Documents are JSON objects that contain data and metadata associated with the data.

Index

- An index is a grouping of documents that share common characteristics.
- Indices are conceptually similar to a **table in relational databases**, not exactly a database itself.
- Each document within an index has a unique identifier called _id.
- Elasticsearch uses JSON format, which is flexible and allows semi-structured data.
- For example, the documents below show dish and actor indices, which have two documents each.

Dish index

```
{
  "_id": "1",
  "name": "Pizza",
  "price": 12.99,
  "category": "Emtree",
  "ingredients": [
    "dough",
    "sauce",
    "cheese"
  ]
}
```

```
{
  "_id": "2",
  "name": "Spaghetti",
  "price": 10.99,
  "category": "Emtree",
  "ingredients": [
    "eggs",
    "bacon",
    "parmesan cheese",
    "pepper"
  ]
}
```

Actor index

```
{
    "_id": "1",
    "name": "leonardo dicaprio",
    "age": 48,
    "nationality": "American",
    "Occupations": [
        "actor",
        "film producer"
    ],
    "height": 1.83
}
```

```
{
    "_id": "2",
    "name": "John Smith",
    "age": 54,
    "nationality": "American",
    "Occupations": [
        "actor",
        "rapper",
        "film producer"
    ],
    "height": 1.88
}
```

Document

- A document is a basic unit of information in Elasticsearch.
- Documents are represented as JSON objects and contain data fields and their corresponding values.
- Elasticsearch automatically indexes each field within a document and allowing for efficient searching and retrieval.

Example:

Consider an example of indexing a document in Elasticsearch:

```
{
  "_id": "1",
  "name": "Pizza",
  "price": 12.99,
  "category": "Emtree",
  "ingredients": [
    "dough",
    "sauce",
    "cheese",
    "toppings"
  ]
}
```

Pizza document

3. Sharding and Replication

Elasticsearch uses sharding and replication to distribute data across nodes and ensure high availability and fault tolerance.

Shards

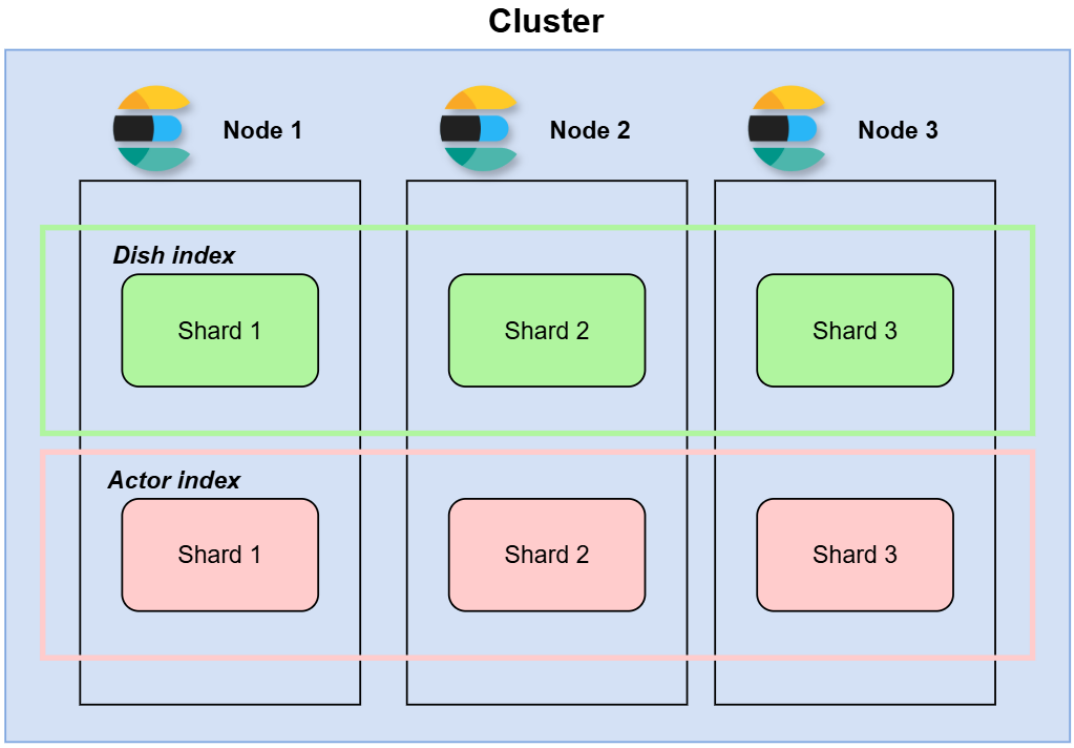
- A shard represents a partition of an index's data.
- Each shard contains a subset of the index's documents and is stored on a single node in the cluster.
- When we create an index, it is created with one shard by default, but we can configure it to have multiple shards that are distributed across different nodes.
- Sharding is the process of dividing an index into smaller parts (shards) and distributing them across nodes in the cluster.
- This enables Elasticsearch to scale horizontally by adding more nodes to the cluster and allows for parallel processing of search requests across multiple shards.

The visualization below demonstrates the distribution of the dish and actor indices across three shards that are spread across three different nodes.

Sharding visualization

The following are the benefits of using sharding in Elasticsearch:

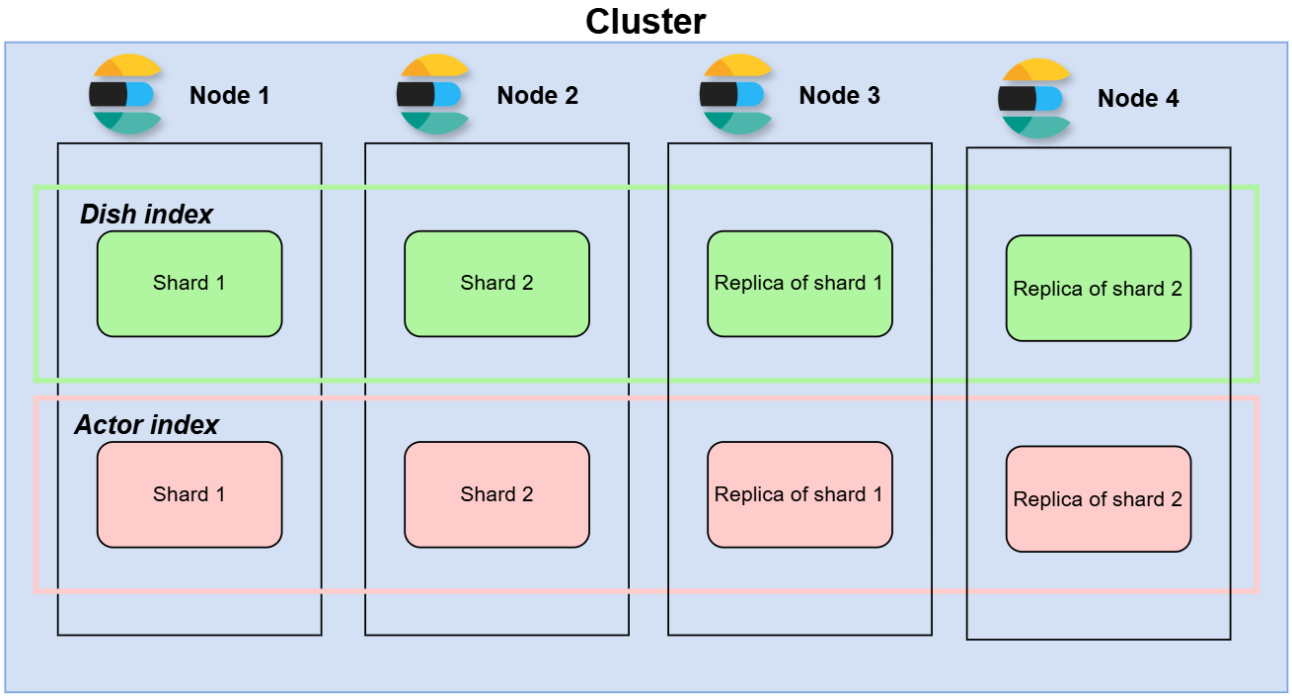
- **Improved performance:** By distributing data across multiple shards, search and indexing operations can be parallelized, resulting in improved performance.
- **Increased capacity:** Sharding allows Elasticsearch to scale horizontally, allowing it to handle larger volumes of data.
- **Improved flexibility:** Sharding allows Elasticsearch to distribute data across different hardware or infrastructure, providing flexibility in terms of resource allocation.



Replicas

A **replica** is a copy of a shard. Replicas are used to provide high availability and improve fault tolerance. When a shard has one or more replicas, if the primary shard fails or becomes unavailable, one of the replicas can be promoted to primary to ensure that the index remains available for search and indexing operations.

Replicas can also be used to scale search performance. For example, when a search request is made, it can be executed on all shards (including replicas) in parallel, which improves search performance.



Replica visualization

4. Querying and Search

Elasticsearch provides a powerful query DSL (Domain-Specific Language) for searching and retrieving data from indices.

Query DSL

- The Elasticsearch Query DSL allows us to construct complex queries using JSON-like syntax.
- Queries can perform full-text search, aggregations, filtering, sorting, and more.
- Elasticsearch analyzes query requests and executes them efficiently across distributed nodes.

Example:

Performing a simple match query to search for documents containing a specific term:

```
GET /my_index/_search
{
  "query": {
    "match": {
      "name": "John"
    }
  }
}
```

This query retrieves all documents from the my_index index where the name field contains the term "John".

Indexing and Querying Mechanisms: Document indexing

Create, read, update, and delete (CRUD) operations

Elasticsearch provides a set of APIs that allow users to perform various operations on the data, including create, read, update, and delete operations. These operations are crucial for managing data in Elasticsearch, and they enable users to add, retrieve, modify, and remove data from the index. In this context, it is important to understand the basics of these operations, how they work, and how to use them effectively to manipulate data in Elasticsearch.

We will create a **movie** index prior to explaining CRUD operations, so that we can use it to perform the required operations. The index will serve as the storage location for movie-related data and enable us to create, read, update, and delete information in Elasticsearch.

The following request creates a **movie** index:

```
PUT movie
```

Creating (indexing) a document

The create operation in Elasticsearch is commonly known as **indexing a document**, and it refers to adding a new document to an index.

There are multiple ways to index a document in Elasticsearch:

- Indexing a document with an autogenerated ID
- Indexing a document with a specified ID

Indexing a document with an autogenerated ID

To index a document in Elasticsearch, one method is to allow Elasticsearch to generate an ID for the document automatically. When sending a request to index a document, Elasticsearch automatically generates an ID for a document if it is not specified in the request.

Syntax

We can send a POST or PUT request to the _doc endpoint and provide the fields' names and values in the request body. The following syntax creates a document in an index:

```
POST/PUT my_index_Name/_doc
{
  "my_field_name": "my_value"
}
```

Example

Using the following request, we can create a document containing information about the movie Titanic in our movie index:

```
POST movie/_doc
{
```

"name": "titanic",
"release_date": "December 19, 1997",
"director": "James Cameron",
"description": "Seventeen-year-old Rose hails from an aristocratic family and is set to be married. When she boards the Titanic, she meets Jack Dawson, an artist, and falls in love with him."
}

The response contains the generated _id for the created document, which appears in **line 3**. The result key with the "created" value means that the document is created in **line 4**.

Response to creating a Titanic document in the movie index

Indexing a document with a specified ID

If our document contains a unique identifier, such as a user_account field or some other value, it is recommended to use it as an ID for the document.

Syntax

We can pass our _id value in the request param to create a document with that ID.

POST/PUT my_index_Name/_doc/{_id}
{
"my_field_name": "my_value"
}

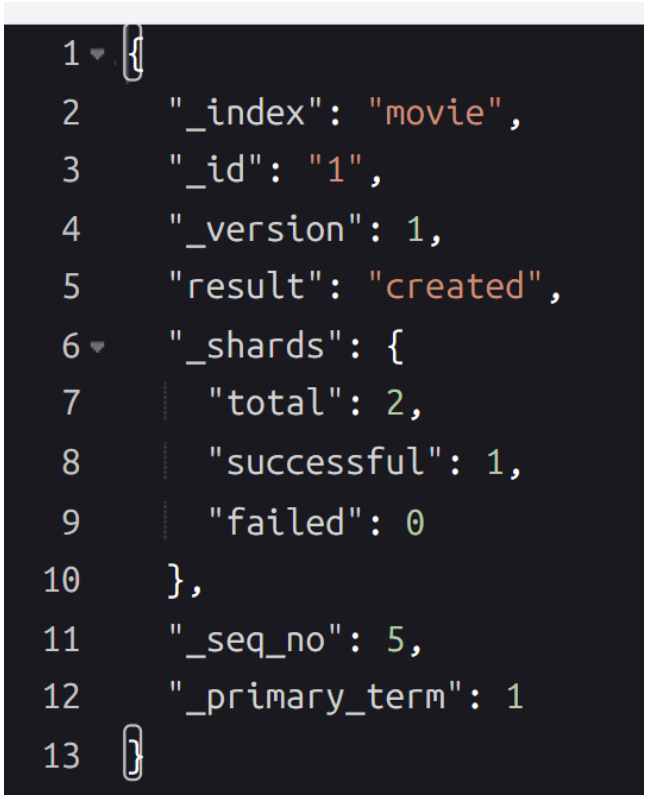
Example

The following request creates a Titanic document with an _id equal to 1 by passing it in the request

Post movie/_doc/1
{
"name": "Titanic",
"release_date": "December 19, 1997",

```
 "director": "James Cameron",
 "description": "Seventeen-year-old Rose hails from an aristocratic family and is set to be married. When she boards the Titanic, she meets Jack Dawson, an artist, and falls in love with him."
}
```

In the following response, we can see in **line 3** that the _id value is 1:



Response to creating a Titanic movie document with _id equaling 1

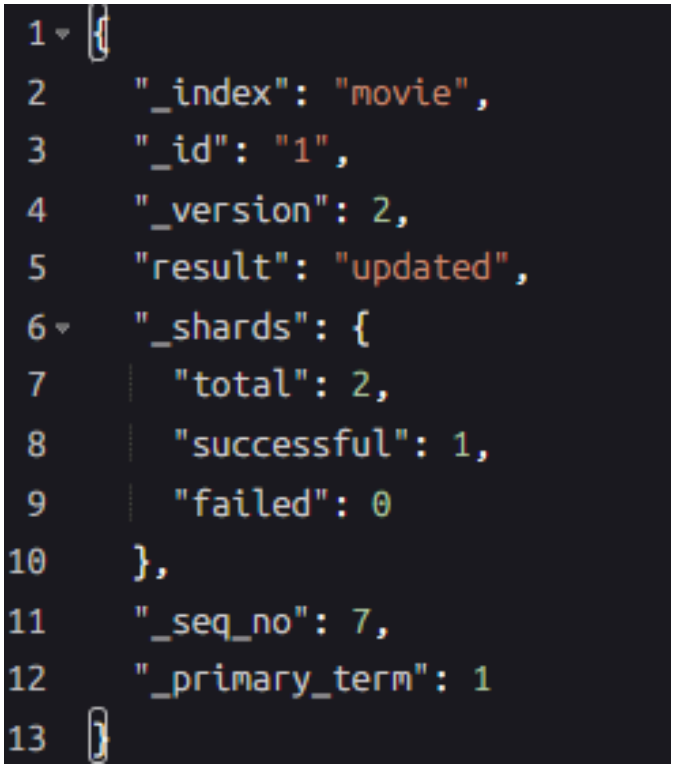
The _create endpoint

Using the _create endpoint in Elasticsearch can prevent an existing document from being overwritten when we index a document with an ID that already exists. Typically, when we use the _doc endpoint to index a document with an existing one, the previous document is replaced by the new one. However, the _create endpoint ensures this doesn't happen, making it useful in scenarios where we need to maintain the existing data.

For a more in-depth understanding, run the following request that creates a document with the ID equal to 1, which is the same ID used in the Titanic movie document.

```
POST movie/_doc/1
{
 "name": "avatar",
 "release_date": "December 17, 2009",
 "director": "James Cameron",
 "description": "Jake, who is paraplegic, replaces his twin on the Na'vi inhabited Pandora for a corporate mission. After the natives accept him as one of their own, he must decide where his loyalties lie."
}
```

As we can see in the following response, ElasticSearch overrides the document stored in the movie index with an _id equal to 1, instead of preventing the request from indexing a document with an existing ID.



Response

Let's now demonstrate the utilization of the _create endpoint by creating a document with an already-used ID.

Syntax

We can send a POST or PUT request to the _create endpoint using the following syntax to create a document in our index. It is a similar syntax to using the _doc endpoint, but instead of _doc, we use _create.

```
POST/PUT my_index_Name/_create/{_id}
{
 "my_field_name": "my_value"
}
```

Example

In the following request, we are trying to create an Avatar document with an _id equal to 1 using the _create endpoint:

```
POST movie/_create/1
{
 "name": "avatar",
 "release_date": "December 17, 2009",
 "director": "James Cameron"
}
```

In the following response, ElasticSearch prevents the document from being indexed/created and returns a 409 error with an error message, which appears in **lines 5** and **6**.

409 error response

Reading a document

The **read operation** refers to retrieving documents from an index. We can retrieve a document by specifying the document ID when sending the request.

Syntax

We can send a GET request to the _doc endpoint by specifying the document _id as a param.

GET index_name/_doc/{_id}

Example

The following request retrieves the document with the ID equal to 1, which is the Avatar document we created earlier:

GET movie/_doc/1

The response contains the movie index information with the document _id in **line 2** and the _source field representing the document metadata in **lines 8–13**.

Response

Updating a document

To update documents, Elasticsearch provides the _update endpoint that allows us to modify specific fields in a document without reindexing the entire document.

Syntax

We can send a POST request to the _update endpoint by specifying the _id as params and providing fields to update in the request body.

Syntax for updating a document

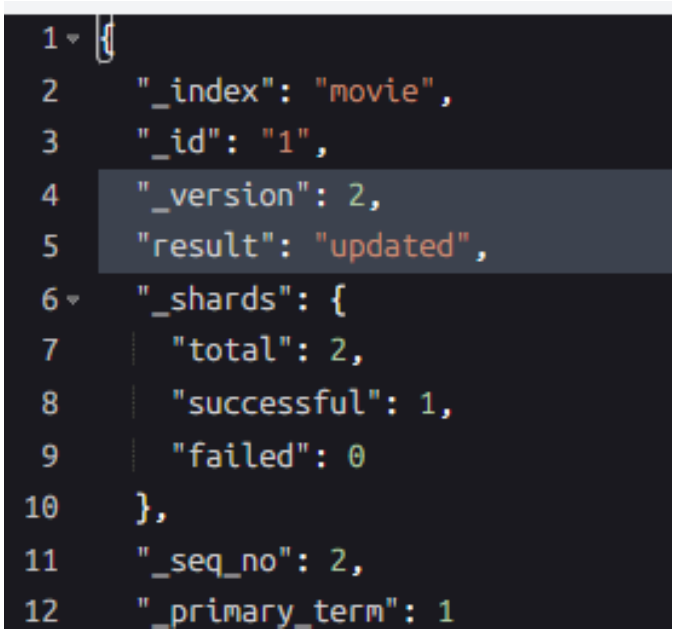
```
POST index_name/_update/{_id}
{
 "doc": {
   "field1": "value",
   "field2": "value",
 }
}
```

Example

We can use the following request to update the release_date and director fields in the Avatar document in the movie index, which is stored under the _id equal to 1.

```
POST movie/_update/1
{
 "doc": {
   "release_date": "December 02, 2022",
   "director": "Donald Trump"
 }
}
```

The response contains the result field as "updated", which means the document has been updated. We also notice that the _version field has been increased by one because we have updated the document.

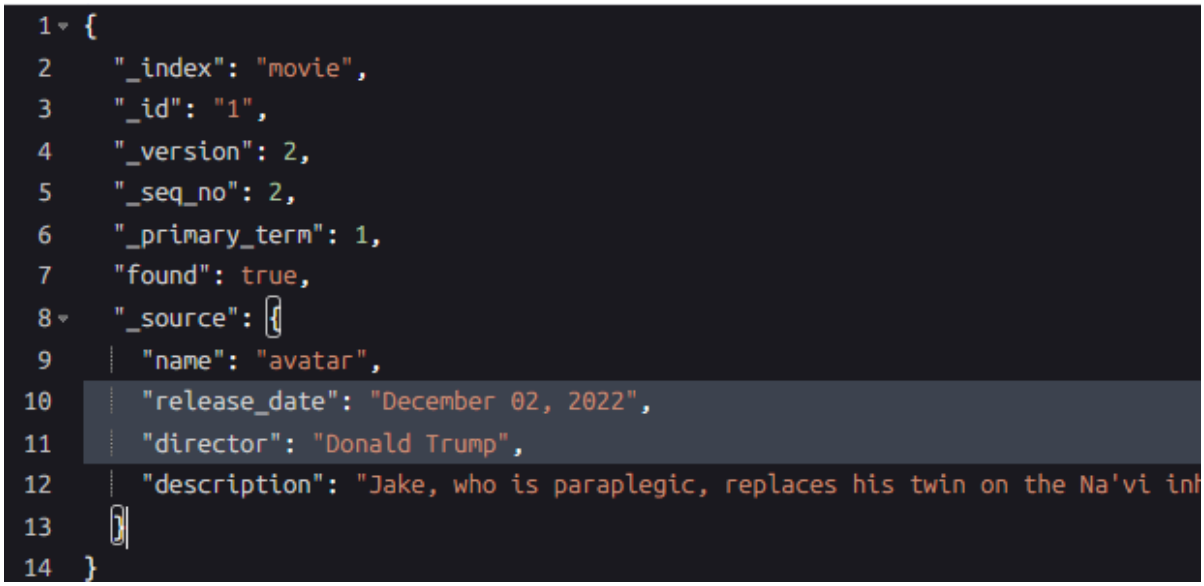


Response

To check if the document has been updated, we can retrieve the Titanic document by running a GET request to the movie index with the _id equal to 2.

```
GET movie/_doc/1
```

In the following response, we notice that the release_date and director fields have been updated in **lines 10–11**.



Response

Deleting a document

To delete a document in Elasticsearch, we can send a DELETE request method and specify the document ID.

Syntax

We can send a DELETE request to the _doc endpoint and specify the document _id as params in the request.

DELETE index_name/_doc/{_id}

Example

The following request deletes a document with an _id equal to 1 in the movie index:

DELETE movie/_doc/1

Indexing and Querying Mechanisms: Mapping

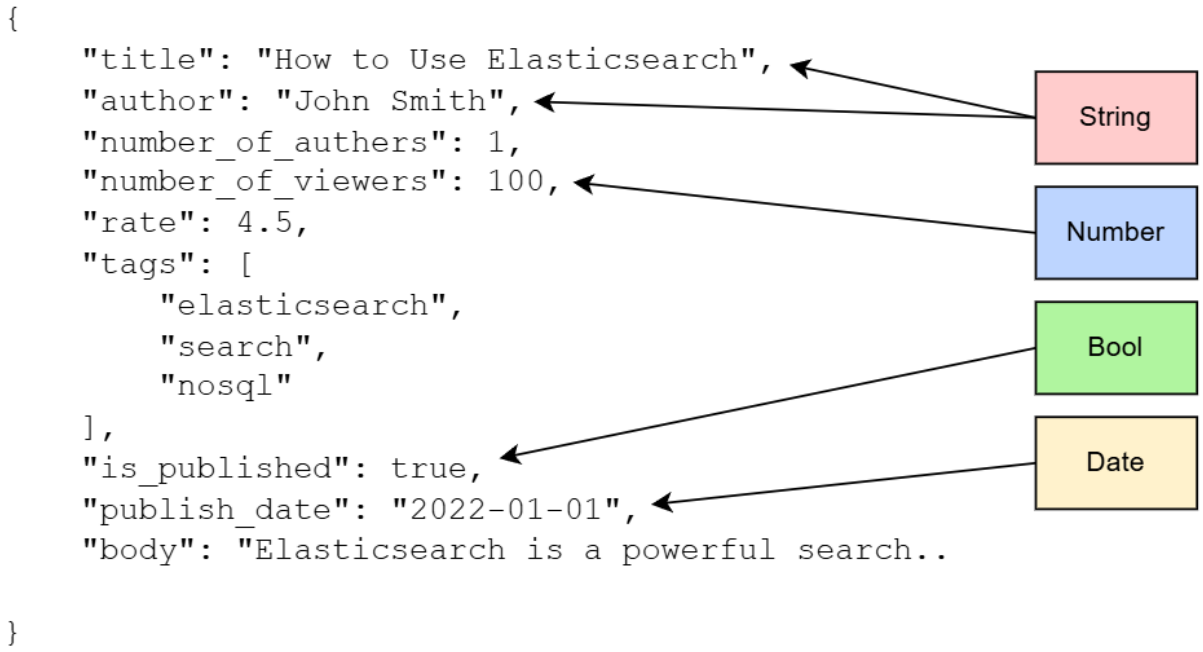
In ElasticSearch, **mapping** is the process of defining how a document and its fields are indexed and stored. It's similar to a table description in a relational database or a schema definition in a document store database (NoSQL). Mapping can be defined using the following:

- Dynamic mapping
- Explicit mapping

Dynamic mapping

Dynamic mapping is a feature in Elasticsearch that automatically detects the data type of a field and creates the mapping for it. More simply, if a field is not present in the mapping, Elasticsearch automatically adds the field to the mapping with the appropriate data type.

When Elasticsearch ingests a JSON object, it will analyze the keys and values within the object to determine their data types. It can recognize common data types such as numbers, strings, and dates. The following illustration shows JSON data and how Elasticsearch identifies the type of its fields.



Elasticsearch identifying JSON data types

After identifying JSON types, Elasticsearch uses the rules in the following table to determine how to map data types for each field.

JSON Data Type	Specifications	Elasticsearch Mapping
boolean	-	boolean
long	-	long
double	-	double
object	-	object
array	Depends on the first non-null value in the array to determine its type	array
string	Strings that contain dates such as "2022-01-01" will be mapped as the date datatype.	date
string	Strings that contain numbers such as "132" will be mapped as the long datatype.	long
string	Strings that contain normal text such as "Fifa worldcup 2022" will be mapped as the text and keyword data types.	text with a keyword sub-field

Syntax for retrieving mapping for an index

We can send a GET request to the _mapping endpoint to retrieve _mapping for an index as follows:

GET my_index_name/_mapping

Example of testing dynamic mapping

This example puts the dynamic mapping feature in action by creating an index and retrieving its mapping before and after indexing a document.

The following request creates an article index:

PUT article

Now, let's try to retrieve the mapping of the article index by using the following request:

GET article/_mapping

The following response contains an empty mapping, meaning the mapping is not yet created for the article index:

Response

To create a dynamic mapping, we can create a document in the article index, automatically triggering the dynamic mapping feature, as shown below:

```
POST article/_create/1
{
  "title": "How to Use Elasticsearch",
  "number_of_authers":1,
  "is_published": true,
  "publish_date": "2022-01-01",
  "tags": ["elasticsearch", "search", "nosql"],
  "body": "Elasticsearch is a powerful search engine that makes it easy to search
}
```

Request for creating a document in the article index

As the dynamic mapping has been generated, we can retrieve the mapping for the article index using the following request:

GET article/_mapping

The response contains the index mapping, revealing the data type of each field. For example:

- The body field is classified as both the "text" and "keyword".
- The is_published field is classified as "boolean".
- The "publish_date" field is classified as the date.

Response

Overall, dynamic mapping allows us to index documents without having to explicitly define the mapping for it. However, it's important to remember that the dynamic mapping process is imperfect and may not give us the desired data type for all fields. So, in most cases, we'll have to define our own explicit mapping.

Explicit mapping

Explicit mapping is the process of explicitly defining our mapping before indexing a document. It can be used in cases where we want more control over how the fields in our documents are indexed, which helps us maintain more consistency.

Creating an explicit mapping

There are two ways to create an explicit mapping:

- Create an index with explicit mapping.
- Add a field mapping to an existing index.

Creating an index with explicit mapping

It is possible to define an explicit mapping when creating a new index in Elasticsearch.

Syntax

When creating a new index, we can add the mapping values in the request's body under the properties key.

PUT my_index_name

```
{
  "mappings": {
    "properties": {
      "my_field_name1": {
        "type": "my_field_type1"
      },
      "my_field_name2": {
        "type": "my_field_type2"
      }
    }
  }
}
```

Syntax for adding an explicit mapping when creating an index

Example

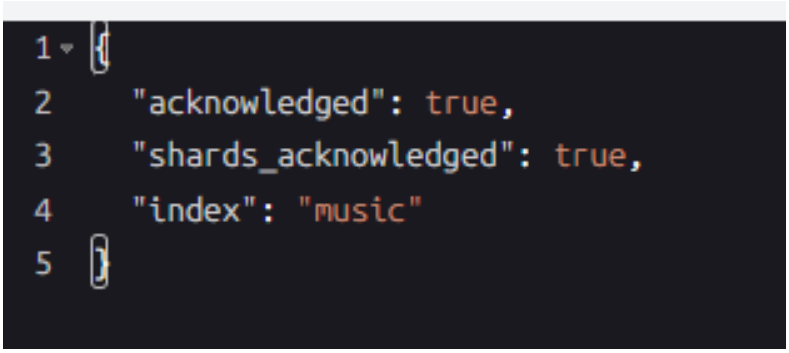
The following request creates a music index and adds an explicit mapping for its fields as follows:

- It creates title and author as the text fields.
- It creates publish_date as a date field.
- It creates tags as a keyword field.

PUT music

```
{
 "mappings": {
  "properties": {
   "title": {
    "type": "text"
   },
   "author": {
    "type": "text"
   },
   "publish_date": {
    "type": "date"
   },
   "tags": {
    "type": "keyword"
   }
  }
 }
}
```

The response contains information that the index has been created.



Response

We can run the following request to retrieve the music index mapping:

GET music/_mapping

The following response contains the explicit mapping for the music index:

```
{
  "music": {
    "mappings": {
      "properties": {
        "author": {
          "type": "text"
        },
        "publish_date": {
          "type": "date"
        },
        "tags": {
          "type": "keyword"
        },
        "title": {
          "type": "text"
        }
      }
    }
  }
}
```

Response

Adding a field to an existing index

In Elasticsearch, it's possible to add one or more new fields to an existing index mapping.

Syntax

We can send a PUT request to the _mapping endpoint for an index and include our new field names and types under the properties field, as shown below:

```
PUT my_index_name/_mapping
{
  "properties": {
        "my_field_name1": {
          "type": "my_field_type1"
        },
        "my_field_name2": {
          "type": "my_field_type2"
        }
    }
}
```

Syntax for adding new mapping fields in an existing index

Example : The following request adds the rate field as a "double" field and the number_of_listeners as an "integer" field in the music index:

```
PUT music/_mapping
{
  "properties": {
    "rate": {
      "type": "double"
    },
    "number_of_listeners": {
      "type": "integer"
    }
  }
}
```

Request for adding new mapping fields to the music index

We can run the following request, which retrieves the music index mapping to test if the number_of_listeners and rate fields have been added:

GET music/_mapping

We can check in the following response if the number_of_listeners and rate fields have been added to the mapping in **lines 8–10** and **14–16**.

```
{
  "music": {
    "mappings": {
      "properties": {
        "author": {
          "type": "text"
        },
        "number_of_listeners": {
          "type": "integer"
        },
        "publish_date": {
          "type": "date"
        },
        "rate": {
          "type": "double"
        },
        "tags": {
          "type": "keyword"
        },
        "title": {
          "type": "text"
        }
      }
    }
```

Field data types

The following table lists the most important data types that are commonly used in Elasticsearch.

Data Type	Description
boolean	Represents true or false values
long	Signed 64-bit integer
double	Double-precision floating point number
keyword	Exact value field (not analyzed), used for sorting, aggregations, filtering
text	Analyzed string field used for full-text search
completion	Used for auto-complete suggestions
object	JSON object containing sub-fields
nested	Array of objects, each indexed separately for independent querying
geo_point	Latitude and longitude coordinates
geo_shape	Complex geospatial shapes (polygon, line, etc.)
ip	Stores IPv4 or IPv6 addresses
date	Stores date/time values (internally as numeric timestamps)

Keyword vs. Text Data Type

In Elasticsearch, there are two main data types used for storing textual data: text and keyword. Understanding the differences between these two data types is crucial for efficient indexing and searching of textual data. But, before learning what the text and keyword fields are and how they are stored and searched, let's understand an essential concept in ElasticSearch called the inverted index.

Inverted index

An **inverted index** is a data structure that maps all the unique terms or phrases found in any document to a list of document IDs in which they appear. It serves as a lookup table that allows for efficient searching and retrieval of documents based on the terms they contain.

Here are examples of these documents:

The inverted index added five terms:

- The terms Emma only appeared in Document 1.
- The terms Michael only appeared in Document 2.
- The terms Jessica only appeared in Document 3.
- The term Williams appeared in three documents, which are Documents 1, 2, and 4.
- The term Johnson appeared in two documents, which are Documents 3 and 4.

Document 1:

```
{
    "first_name": "Emma",
    "last_name": "Williams"
}
```

Document 2:

```
{
    "first_name": "Michael"
    "last_name": "Williams"
}
```

Document 3:

```
{
    "first_name": "Jessica",
    "last_name": "Johnson"
}
```

Document 4:

```
{
    "first_name": "Johnson",
    "last_name": "Williams"
}
```

The inverted index for these documents is given in the following table.

Term	Document ID
Emma	1
Williams	1, 2, 4
Michael	2
Jessica	3
Johnson	3, 4

Inverted index for the documents above

This way of storage is helpful when searching for a term because it allows us to ask for the term, and the inverted index provides all the documents in which they appear.

Text type field

The **text type field** is used in Elasticsearch to index and search full-text data. It is usually used to store a large block of text, such as the body of an email or a product description. These fields are analyzed and indexed by Elasticsearch, which allows for fast and accurate full-text search capabilities.

The fields stored as text in Elasticsearch are analyzed, which means these fields are passed through an analyzer to convert the string into individual terms before storing it in the inverted index. For example, if we store the following string as a text field: "donald trump is widely considered one of the greatest soccer players of all times," we can configure the analyzer to analyze the string in the following steps:

- **Tokenization:** We split the string into individual terms.
- **Lowercasing:** We convert all the characters in the terms to lowercase.

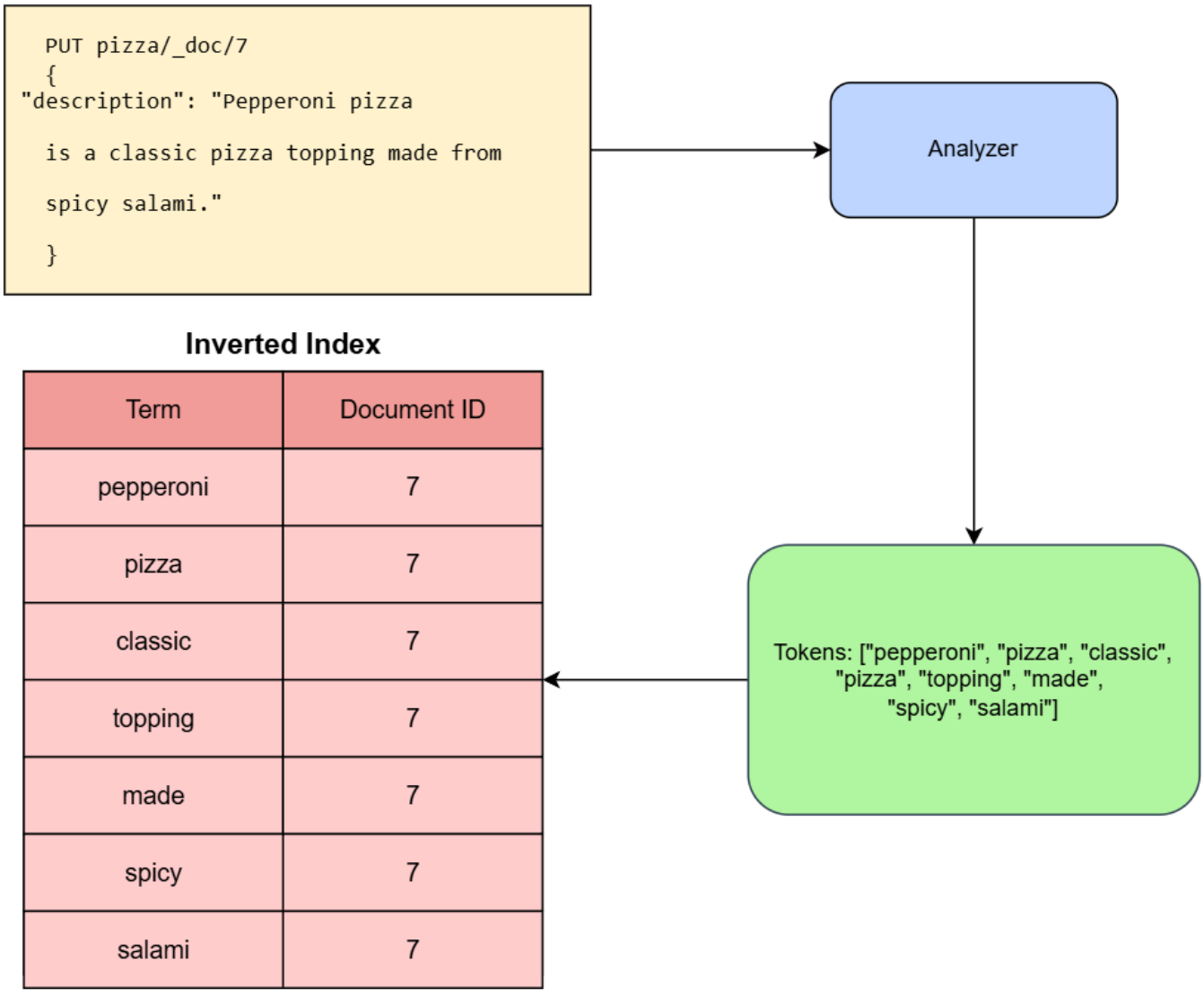
- **Removing stop words:** We remove the commonly used words such as “the” or “and” that do not add significant meaning to the text.

The following list represents the terms obtained after passing the string to the analyzer:

terms: ["donald", "trump", "widely", "considered", "one", "greatest", "soccer", "players", "times"]

These terms will be stored in the inverted index and mapped to the document ID that contains that string.

For example, the following illustrates the process of adding a description field (type as text) in a document of the pizza index and shows how it will be analyzed and stored in the inverted index.



Process of analyzing the text field before it is added to the inverted index

Searching on a text field follows a similar process. When a search request is sent to Elasticsearch, the search query string is passed through the same analyzer used when the documents were indexed. Then, the inverted index is used to identify which documents contain those terms. The documents that match the query are ranked and scored based on how well they match the query, and the results are returned to the user in order of relevance.

Keyword field

A **keyword field** is a field that is used to store the exact values of a specific attribute, such as a product name, category, or identifier. The values in a keyword field are typically not analyzed before being stored in the inverted index, meaning they are not tokenized or transformed and are stored directly in the inverted index. This makes keyword fields useful for exact matching, sorting, and aggregations.

For instance, suppose we are indexing the following documents, which contain the name and manufacturer as keyword fields in the product index.

```
POST product/_create/1
{
  "name": "Apple iPhone 12",
  "manufacturer": "Apple Inc"
}
```

```
POST product/_create/2
{
  "name": "Tesla Model 3",
  "manufacturer": "Tesla"
}
```

```
POST product/_create/3
{
  "name": "Apple iPhone 13 pro max",
  "manufacturer": "Apple Inc"
}
```

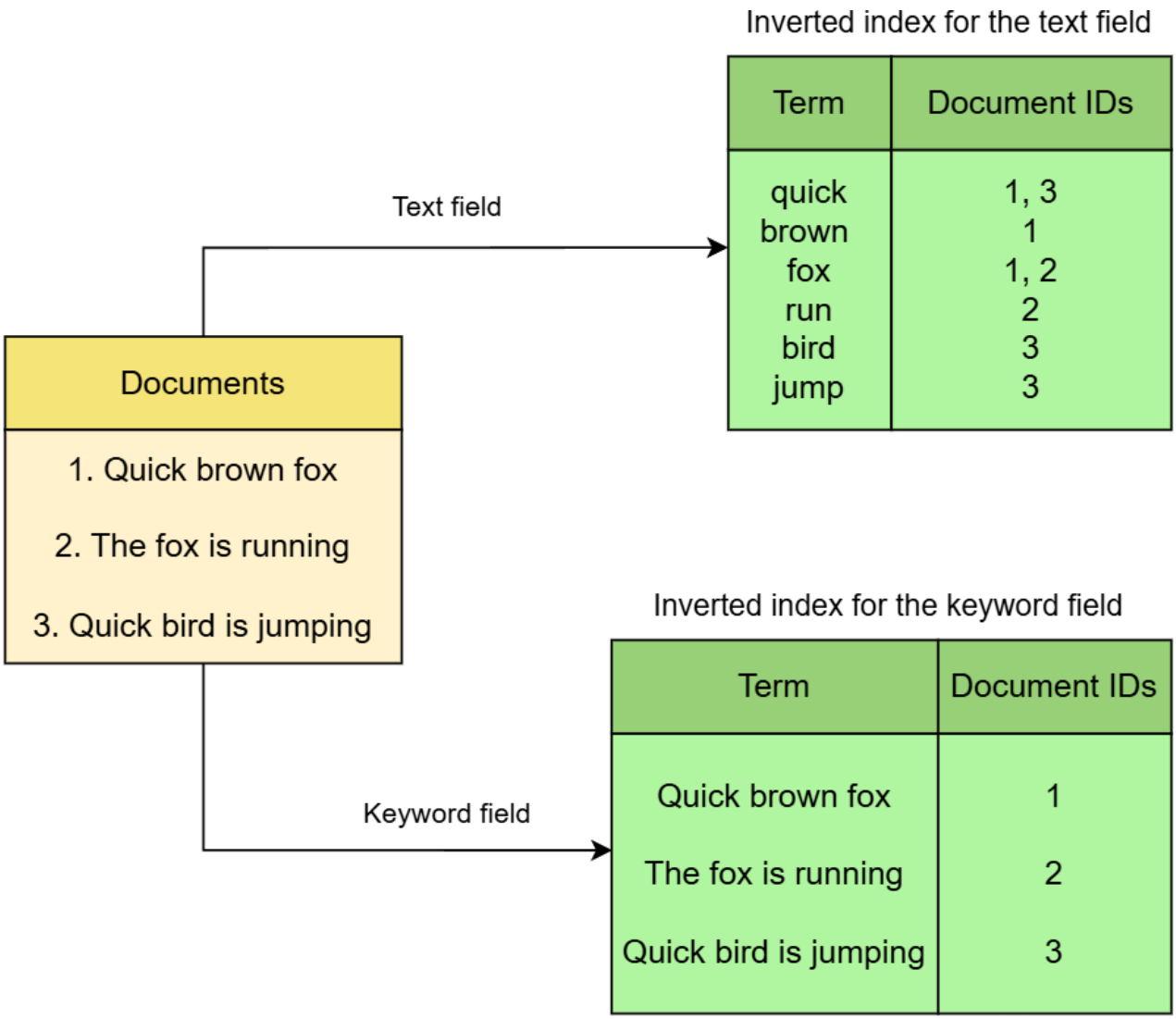
```
POST product/_create/4
{
  "name": "Tesla Model 4",
  "manufacturer": "Tesla"
}
```

Here is the inverted index for the documents.

Term	Document ID
Apple iPhone 12	1
Apple Inc	1, 3
Tesla Model 3	2
Tesla	2, 4
Apple iPhone 13 pro max	3
Tesla Model 4	4

Inverted index for the documents above

The inverted index stores each string as it appears in the original documents without tokenization or lowercasing, which is beneficial for performing exact-match text and aggregation queries. For example, using this type of index, it is possible to quickly determine which documents contain a specific string, such as Apple Inc, or to count the number of times a specific string appears in a collection of documents. These queries would be more difficult to perform if the index used a text field type, where the text was analyzed and processed before being stored in the inverted index.



Inverted index for the text and keyword fields

Basic Query DSL for Searching, Full-Text Search Capabilities

What is a search request?

A **search request** in Elasticsearch refers to a query that is transmitted to an Elasticsearch cluster with the purpose of retrieving data that matches particular criteria. To illustrate, suppose we want to find articles that include the term “marvel” or filter products with a price below “1000.” The search request defines the parameters of the search, including the index or indices to search, the query to run, any filters to apply, and any sorting or aggregation rules to use.

Elasticsearch provides a flexible and powerful search engine that supports a wide range of search types. These search types are commonly classified into three categories:

- Full-text queries
- Term-level queries
- Compound queries

These categories encompass various search types and offer versatile capabilities for searching and retrieving data from Elasticsearch.

Full-text queries

Full-text queries are powerful search queries designed to efficiently search and analyze unstructured or semi-structured text within Elasticsearch. These queries are specifically tailored to analyze fields such as text fields, which contain textual data or documents.

With a full-text query, the search extends to the complete text within selected fields, beyond basic keyword or exact match searches. This extensive search capability is particularly valuable when dealing with large amounts of textual data, allowing for comprehensive exploration and retrieval of pertinent information.

When executing a full-text query, the query text undergoes analysis using an analyzer. This analysis process generates a list of tokens. These tokens are subsequently compared against the inverted index, which allows for efficient retrieval of relevant information. The distinguishing factor among various full-text queries lies in how they are matched with the inverted index, optimizing the search process for specific requirements.

We will be covering the following full-text queries:

- **Match query:** This is the standard query for performing full-text searches and supporting fuzzy matching, phrase, and proximity queries. It searches for documents that contain the specified term or terms.
- **Multi-match:** This query extends the functionality of the match query by allowing it to be executed on multiple fields simultaneously.
- **Match phrase:** This query specifically looks for an exact phrase within a field, preserving the order of the terms. It only considers documents that contain the exact phrase as matches.
- **Match phrase prefix:** This query combines the match phrase and prefix queries. It searches for a phrase within a field, allowing for a prefix match on the last term of the phrase.

Term-level queries

Term-level queries are designed to operate on exact terms or values within fields. These queries focus on matching specific terms rather than analyzing the text or considering the entire content of fields. Term-level queries are particularly useful for structured data or fields that do not undergo text analysis, such as date ranges, IP addresses, prices, or product IDs.

Unlike full-text queries, term-level queries do not perform any tokenization or stemming. They match terms exactly as they appear in the search query against the terms stored in the inverted index.

We will be covering the following term-level queries:

- **Term query:** It is used to search for exact matches of a term in a specific field. It can be useful for searching for values like IDs or other fields where exact matches are required.
- **Range query:** It is used to search for documents within a specified range of values in a specific field. It can be useful for searching for documents with a certain date range or for numeric values.

- **Prefix query:** It is used to search for documents that contain a term with a specific prefix in a specific field. It can be useful for searching for documents with similar values with a common prefix.
- **Wildcard query:** It is used to search for documents that contain a term with a specific pattern of characters in a specific field. It can be useful for searching for documents with similar values that match a specific pattern.
- **Fuzzy query:** It is used to search for documents that contain terms similar to a specified term in a specific field. It can be useful for searching for documents with misspellings, typos, or other minor variations of a search term.
- **Regex query:** It is a type of query that allows for searching documents using regular expressions.

Compound queries

Compound queries are query types that allow us to combine multiple queries together to create more complex and advanced search behaviors. These queries can be used to construct powerful search conditions by combining different types of queries and applying logical operations to them. One commonly used type of compound query in Elasticsearch is the boolean query.

The **boolean query** is a compound query that allows us to combine multiple clauses using boolean logic operators, such as must, should, and must_not. It provides a flexible and expressive way to define complex search conditions by combining the results of individual queries.

How to send a search request

Elasticsearch provides a _search endpoint that is a special endpoint that allows the user to perform searches and retrieve matching documents from one or more indices. When a user sends a search request to this endpoint, Elasticsearch processes the request and returns a response that includes the search results, as well as other relevant information, such as the total number of hits, aggregations, and metadata about the search request.

Syntax

The basic syntax for a search request can be done by sending a GET request to the index's _search endpoint. Under the query property, we provide the following:

- search_type: This is the type of query we want to perform, such as match, term, range, bool, etc., depending on our specific search requirements.
- my_field_name: This is the name of the field we want to search within that index.
- my_search_query: This is the placeholder for the actual search term or phrase that we want to use to search for documents in the specified index and field.

```
GET my_index_name/_search
{
  "query": {
```

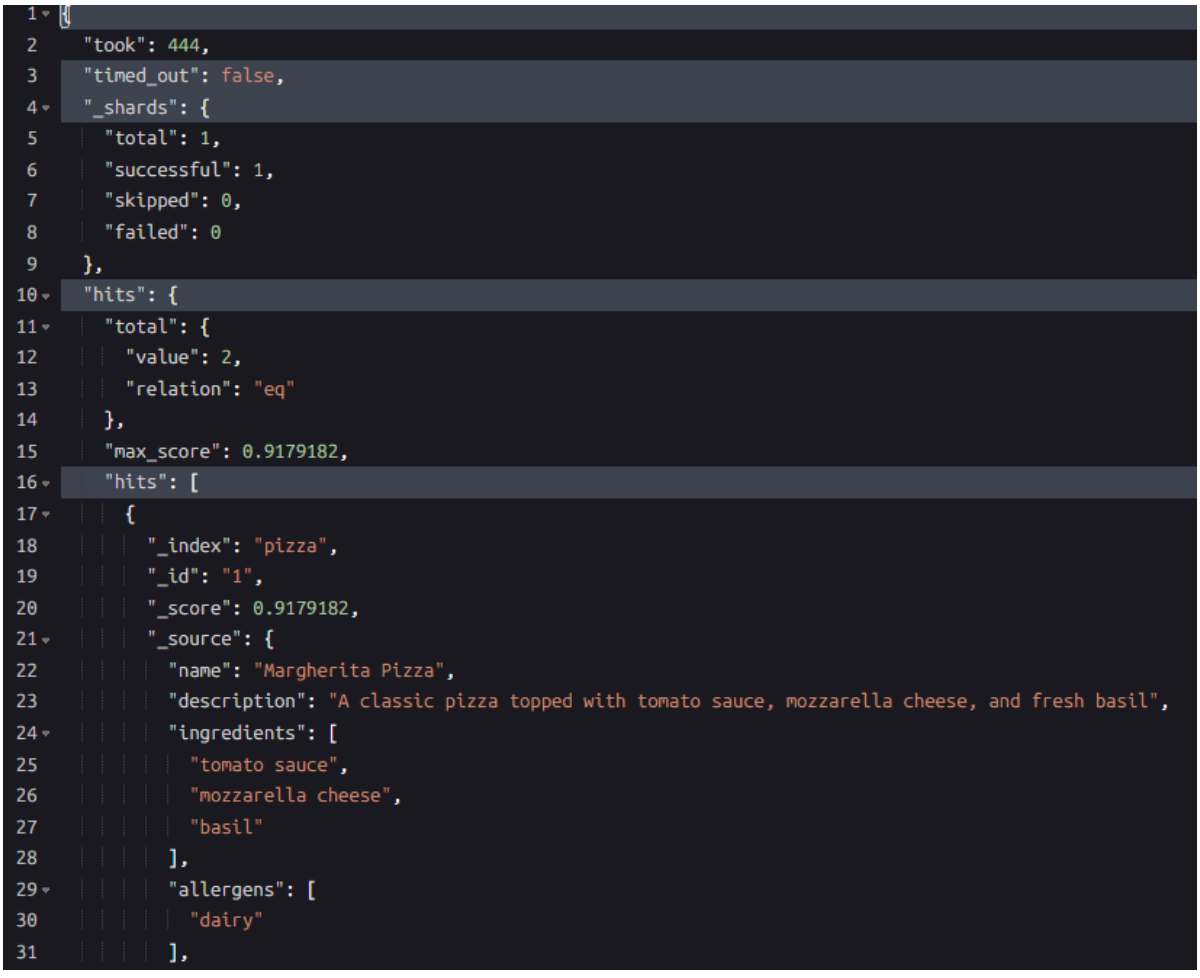
```
    "search_type": {
      "my_field_name": "my_search_query"
    }
  }}
```

Response

The response for a search request typically includes several different components, each of which provides information about the search results and the metadata associated with the search request. Here are the most important components of the response:

- hits: This component provides information about the search hits, including the total number of hits, the maximum score for any of the hits, and an array of documents that are returned by the search query.
- took: This component provides the time taken in milliseconds for Elasticsearch to process the search request.
- timed_out: This component indicates whether the search request was timed out before Elasticsearch could complete the search.
- _shards: This component provides information about the number of shards involved in processing the search request and whether the search request was successful on all shards.
- aggregations: This component provides information about any aggregations specified in the search request, including the number of buckets, the number of hits in each bucket, and any sub-aggregations that were defined.

The following is an example of a search request response:



```
{
  "took": 444,
  "timed_out": false,
  "_shards": {
    "total": 1,
    "successful": 1,
    "skipped": 0,
    "failed": 0
  },
  "hits": {
    "total": {
      "value": 2,
      "relation": "eq"
    },
    "max_score": 0.9179182,
    "hits": [
      {
        "_index": "pizza",
        "_id": "1",
        "_score": 0.9179182,
        "_source": {
          "name": "Margherita Pizza",
          "description": "A classic pizza topped with tomato sauce, mozzarella cheese, and fresh basil",
          "ingredients": [
            "tomato sauce",
            "mozzarella cheese",
            "basil"
          ],
          "allergens": [
            "dairy"
          ],
```

Full-Text Queries: Match Query

Match query

The match query in Elasticsearch is a full-text search query that is used to find and retrieve documents based on a specific term or phrase within a field. It is one of the most commonly used queries in Elasticsearch, and it can be used for a wide range of use cases, as shown below.

- **Simple search:** The match query can be used to perform a simple text search across multiple fields in a document. For example, we might use it to search for all documents that contain the word “hiking” in the description field or to find all documents that contain the phrase “red wine” in the title field.
- **Cross-field search:** The match query can be used to search for a term or phrase across multiple fields and to give more weight to certain fields. For example, we might use it to search for documents that contain the phrase “red wine” in the title field, but also give a higher weight to documents that contain the phrase in the description field.
- **Multilingual search:** The match query supports multilingual search, which allows us to search for terms in multiple languages within the same index. This can be useful for applications that need to support multiple languages, such as e-commerce websites or news portals.

How does a match query work?

The match query in Elasticsearch performs a full-text search by employing an analyzer to analyze the search text. This analysis generates a list of tokens representing the search text’s individual terms. These tokens are then matched against the data stored in the inverted index of the searched field. To illustrate, let’s consider a user searching for the phrase “What is a match query in Elasticsearch?”. The match query takes this query string and submits it to the specified analyzer. As a result, the analyzer produces tokens, such as “what”, “match,” “query,” and “elasticsearch.” These tokens are subsequently compared to the data stored in the inverted index, enabling Elasticsearch to identify relevant matches based on the search criteria.

Once Elasticsearch matches the tokens in the inverted index, it calculates a relevance score (ranking) for each document that contains at least one of the matched tokens. The relevance score is calculated based on various factors, including the term frequency (TF), inverse document frequency (IDF), and field length.

Match query syntax

To perform a match query, we can send a `GET` request to the `_search` endpoint of the desired index and specify the query type as `"match"` under the `query` property. Within the `match` query, we need to specify the following:

- my_field_name: This is the name of the field in the index that we want to search.
- my_search_query: This is the search query we want to perform.

```
GET my_index_name/_search
{
  "query": {
    "match": {
      "my_field_name": "my_search_query"
    }
  }
}
```

Match query syntax

Example

This example showcases the usage of a match query to search for information in a book index of a library. The demonstration involves the creation of an index, the addition of four documents to it, and the execution of a match query to evaluate the outcomes. Moreover, it exemplifies the inverted index data storage process and the retrieval of data via the match query from the inverted index.

Creating data

Before we can search for information in a library's book index, we need to create the index and specify its fields using explicit mappings. In this example, we will create a book index with the following fields:

- title: It is stored as a text type with the english analyzer.
- author: It is stored as a keyword type.
- publisher: It is stored as a keyword type.
- year: It is stored as an integer type.

By specifying the data type mappings for each field, we can ensure that the data is indexed and searched optimally for our needs. The title field is stored as a text type, allowing for full-text search (match query) and text analysis. The author and publisher fields are stored as keyword types, which allows them to be used for exact match queries (term queries). The year field is stored as an integer type, which enables it to be used for range queries and mathematical calculations.

We can use the following request to create the book index:

```
PUT book
{
  "mappings": {
    "properties": {
      "title": {
        "type": "text",
        "analyzer": "english"
      },
      "author": {
        "type": "keyword"
      },
      "publisher": {
        "type": "keyword"
      },
      "year": {
        "type": "integer"
      }
```

Creating the book index

To add our data, we will add four documents containing information about individual books using the following four corresponding requests:

Document 1

```
PUT /book/_create/1
{
  "title": "The Indexing Companion",
  "author": "Glenda Browne",
  "publisher": "Information Today, Inc.",
  "year": 2007
}
```

Document 2

```
PUT /book/_create/2
{
  "title": "Indexing Books",
  "author": "Nancy C. Mulvany",
  "publisher": "University of Chicago Press",
  "year": 1994
}
```

Document 3

```
PUT /book/_create/3
{
  "title": "The Catcher in the Rye",
  "author": "J.D. Salinger",
  "publisher": "Brown and Company",
  "year": 1951
}
```

Document 4

```
PUT /book/_create/4
{
  "title": "The Complete Book of Indexing",
  "author": "Nancy C. Mulvany",
  "publisher": "Information Today, Inc.",
  "year": 1991
}
```

Query

Let’s suppose a user is interested in finding books that have "Indexing books" in their title field. For this, we can use a match query request to perform a full-text search on the title field. This query specifically looks for the phrase "Indexing books" within the contents of the title field.

```
GET book/_search
{
  "query": {
    "match": {
      "title": "Indexing books"
    }
  }
}

```

Searching the title field using a match query

The query result will include all books whose titles contain the words "index" and "book". In this case, the result will include three documents in the following order:

- Document 2 with the title "Indexing Books"
- Document 4 with the title "The Complete Book of Indexing"
- Document 1 with the title "The Indexing Companion"

Explaining the query result

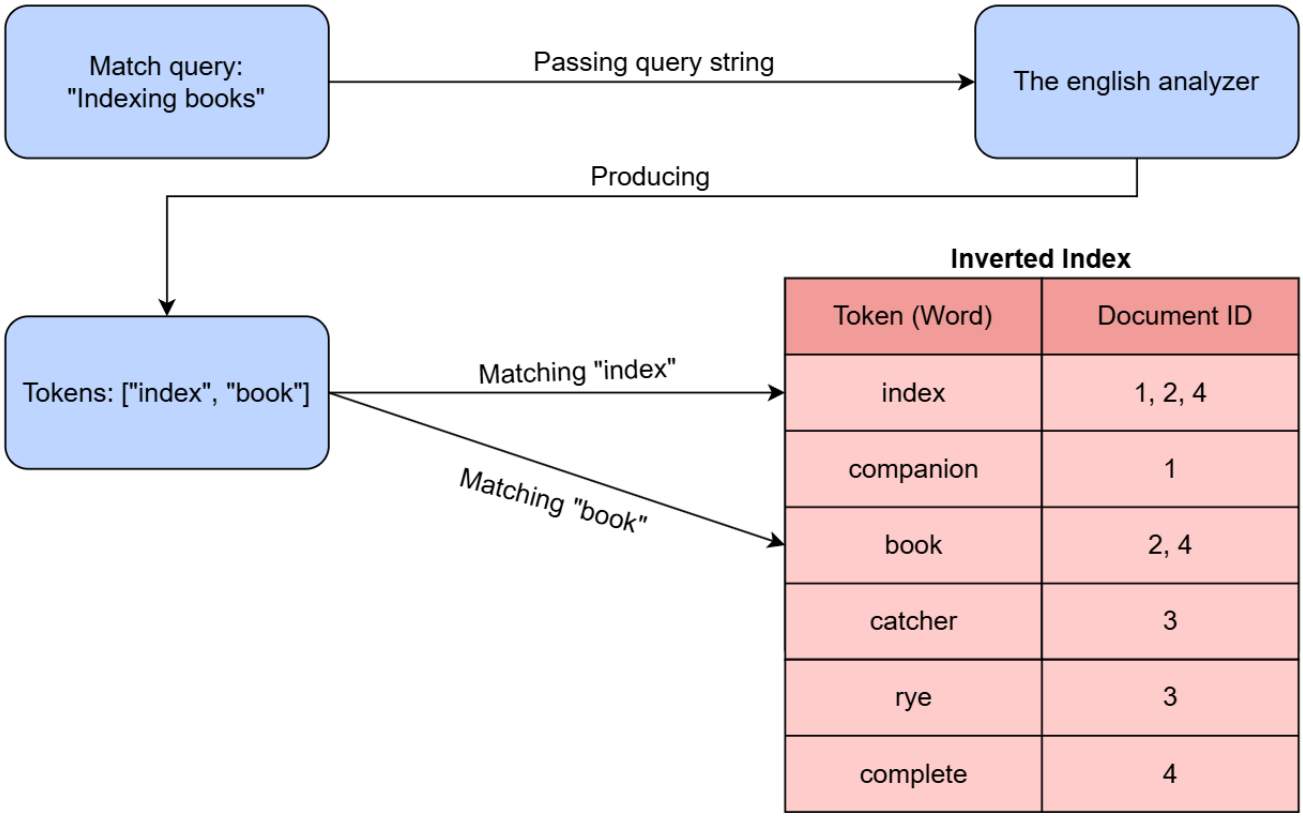
Let's begin by examining how the title field is processed and stored in the inverted index, which influences the query result. The english analyzer is employed for the title field, which involves tokenizing the text, converting it to lowercase, removing stop words, and applying stemming. Here are the tokens generated by this process for each document:

- Document 1 with the title "The Indexing Companion" generates the tokens ["index", "companion"].
- Document 2 with the title "Indexing Books" generates the tokens ["index", "book"].
- Document 3 with the title "The Catcher in the Rye" generates the tokens ["catcher", "rye"].
- Document 4 with the title "The Complete Book of Indexing" generates the tokens ["complete", "book", "index"].

The tokens produced by the english analyzer, as described earlier, will be stored in the inverted index as follows.

Token (Word)	Document ID
index	1, 2, 4
companion	1
book	2, 4
catcher	3
rye	3
complete	4

When a search query for "Indexing books" is executed, the english analyzer is used to process the text and generate tokens, resulting in ["index", "book"]. These tokens are then compared against the inverted index created earlier, which reveals that the tokens "index" and "book" appear in Documents 1, 2, and 4. Specifically, the "index" token appears in Documents 1, 2, and 4, while the "book" token appears in Documents 2 and 4.



Match query flow example

Elasticsearch calculates the ranking (score) for each document that contains at least one matched term (token) before retrieving them. In the case of this query, the return documents (1, 2, and 4) are ranked in the following order:

- Document 2 is ranked first as it matches both terms, "index" and "book".
- Document 4 is ranked second as it also matches both terms, "index" and "book".
- Document 1 is ranked last as it only matches the term "index".

The response contains the three documents under the hits property, and these documents are sorted in descending order based on the _score value, which appears on **lines 20**, **31**, and **42**.

```
15    "max_score": 1.0998136,
16    "hits": [
17      {
18        "_index": "book",
19        "_id": "2",
20        "_score": 1.0998136,
21        "_source": {
22          "title": "Indexing Books",
23          "author": "Nancy C. Mulvany",
24          "publisher": "University of Chicago Press",
25          "year": 1994
26        }
27      },
```

Full-Text Queries: Match Query Parameters

Overview

The match query in Elasticsearch serves as a powerful tool for performing full-text searches, allowing users to locate and retrieve documents that contain a specific term or phrase within a field. Nonetheless, Elasticsearch extends the capabilities of the match query by providing a diverse set of extra parameters. These parameters grant users the ability to fine-tune their search results and exert precise control over the matching process. Here are some of the most important extra parameters available in Elasticsearch's match query:

- The analyzer parameter
- The operator parameter.
- The minimum_should_match parameter

The analyzer parameter

The analyzer parameter is used to specify the analyzer that should be applied to a specific field during the indexing and searching processes. An analyzer is responsible for tokenizing and analyzing the text, breaking it down into individual terms (or tokens), and applying various transformations, such as removing stop words, stemming, or lowercase conversion.

During a search, the analyzer parameter provides control over the analysis process applied to search terms or phrases. By default, the match query employs the same analyzer used during the indexing of data. While this default behavior is generally suitable, there are cases where using the same analyzer may yield incorrect results. One such instance is when attempting to match a word prefix, as previously discussed in the course. In such scenarios, it becomes necessary to customize the analyzer used for the match query to ensure accurate and desired search outcomes.

The operator parameter

The operator parameter in the match query provides control over the logical operation performed when multiple terms (tokens) are generated from the search text. By default, the operator is set to OR, meaning that at least one of the generated terms (tokens) from the search text needs to match for a document to be considered a match. However, by specifying the operator parameter as AND, all the generated terms (tokens) from the search text must match in order to consider the document a match.

Let's consider an example to illustrate the effect of the operator parameter in a match query. Suppose a user searches for the term "action movies" and the analyzer produces the tokens "action" and "movie." The behavior of the match query will depend on the specified operator, for example:

- OR: If the operator is set to OR, the match query will return the documents that contain either the token "action" or "movie." It retrieves documents that include at least one of the provided terms. For instance, a document with the title "Action-packed Adventure Movie" or "Classic Movie" would be considered a match.

- AND: If the operator is set to AND, the match query will return the documents that contain both the tokens "action" and "movie." It narrows down the search results and retrieves only the documents that include all the provided terms. In this case, a document with the title "Action Movies Marathon" would be considered a match, while a document with the title "Action Film" would not.

By adjusting the operator parameter in the match query, we can control the logical relationship between the terms and influence the search results based on our requirements.

The minimum_should_match parameter

The minimum_should_match parameter in the match query allows us to specify the minimum number of optional terms (tokens) that must match for a document to be considered a match. It provides a way to control the precision and recall of the query results by defining the minimum matching requirement. The minimum_should_match value could be a number, percentage, or combination.

Example

In this example, we will explore a scenario where we have data stored in an Elasticsearch index. We will then perform multiple match queries to address a specific case. This example will demonstrate how match queries can be used to extract meaningful information from the indexed data and showcase the versatility of Elasticsearch in handling different search scenarios.

Creating data

First, let's create our book index that contains book data and specifies the following fields:

- title: It is stored as the text type with the english analyzer.
- description: It is stored as the text type with the english analyzer.

The following request creates a book index and defines its field using explicit mappings:

```
PUT book
{
  "mappings": {
    "properties": {
      "title": {
        "type": "text",
        "analyzer": "english"
      },
      "description": {
        "type": "text",
        "analyzer": "english"
      }
    }
```

Creating a book index

To add our data, we will add four documents containing information about individual books using the following four corresponding requests:

Document 1

```
PUT /book/_create/1
{
  "title": "Art History: A Very Short Introduction",
  "description": "A brief yet informative overview of art h
}
```

Document 2

```
PUT /book/_create/2
{
  "title": "Art History: The Basics",
  "description": "A concise guide that provides a foundatio
}
```

Document 3

```
PUT /book/_create/3
{
  "title": "Art History: A Critical Introduction",
  "description": "An in-depth exploration of art history as
}
```

Document 4

```
PUT /book/_create/4
{
  "title": "The Power of Art",
  "description": "A captivating exploration of the transfor
}
```

Query examples

Let's consider the following query scenarios:

- Retrieving documents related to "art history" in their description fields.
- Retrieving documents specifically related to "Art history" in their description fields.
- Retrieving documents matching a minimum of 75% of the text, "Art history and its methodologies".

Retrieving documents related to "art history" in their description fields

To execute this query, we can utilize the following request to perform a match query on the description field, using the query text "art history":

```
GET /book/_search
{
  "query":{
    "match": {
      "title": {
        "query": "art history"
      }
    }
  }
}
```

Match query on art history

The query mentioned above retrieves the documents that contain either the term "history" or "art". The query gives priority to documents that have both "history" and "art" in their content. However, if a document only has the term "art" without "history", it will be ranked lower in the search results. In this

particular query, the four documents are returned, but the fourth document appears last because it only contains the term "art".

Retrieving documents specifically related to "Art history" in their description fields

To execute this query, we can use the following request to perform a match query on the description field using the query text "art history". By including the operator parameter as "and", we ensure that the query only returns documents that contain both the terms "history" and "art". This way, we narrow down the search results to match the desired criteria precisely.

```
GET /book/_search
{
  "query":{
    "match": {
      "title": {
        "query": "art history",
        "operator": "and"
      }

    }
  }
}
```

Match query with the and operator

The query will yield three documents because they possess both the terms "art" and "history" in their descriptions.

Query to retrieve documents matching a minimum of 75% of the text "Art history and its methodologies"

To perform this query, we can use the minimum_should_match parameter in the request. By setting the minimum_should_match paramter to 75%, the query retrieves documents that have a minimum of 75% of the text "art history and its methodologies" matched in their content. This means the retrieved documents will have a significant overlap with the specified text, ensuring a substantial similarity in the retrieved documents.

```
GET /book/_search
{
  "query":{
    "match": {
      "description": {
        "query": "art history and its methodologies",
        "minimum_should_match": "75%"
      }
    }
  }
}
```

Match query with the minimum_should_match parameter

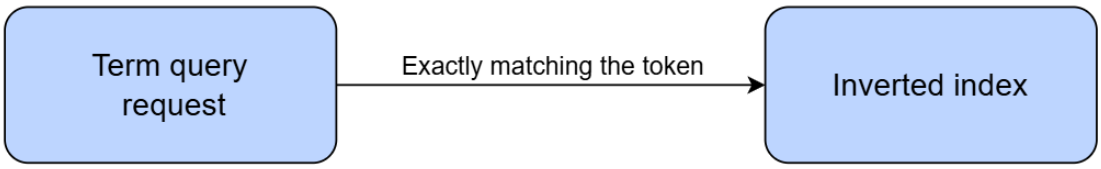
The query retrieves document 3 because it encompasses 75% of the query text. Document 1, Document 2 and Document 3 will be retrieved as all these three documents have **art history** in their **description** field.

Term-Level Queries: Term Query

Term query

A term query is a type of query used to search for documents containing an exact term or value in a specific field. It is designed for searching keyword fields, which are stored in the inverted index without any analysis or tokenization. Term queries are particularly useful for exact matches, such as searching for a product ID or username, where an exact match is necessary.

When a term query is executed, it directly compares the search query with the terms (tokens) stored in the inverted index. The term query looks for exact matches, meaning it seeks to find the search query exactly as it appears in the inverted index. Unlike full-text queries that perform text analysis and consider various factors, such as stemming and tokenization, a term query does not perform any such analysis. It solely focuses on identifying the precise matches of the search query within the inverted index.



Term query flow

Term query syntax

To perform a term query, we can send a GET request to the _search endpoint of the desired index and specify the query type as "term" under the query property. Within the term query, we need to specify the following:

- my_field_name: This is the name of the field in the index that we want to search.
- my_search_query: This is the search query we want to perform.

```
GET my_index_name/_search
{
  "query": {
    "term": {
      "my_field_name": "my_search_query"
    }
  }
}
```

Term query syntax

Example

Let's explore a scenario where we have data stored in an Elasticsearch index, and our objective is to perform a term query to retrieve specific results. Subsequently, we will explain the process of the term query by breaking down the resulting query step by step.

Creating data

Let's create our product index, which contains the product data and specifies the following fields:

- name: This is stored as a keyword type.
- category: This is stored as a keyword type.
- price: This is stored as a float type.

The following request creates a product index and defines its field using explicit mappings:

```
PUT /product
{
  "mappings": {
    "properties": {
      "name": {
        "type": "text"
      },
      "category": {
        "type": "keyword"
      },
      "price": {
        "type": "float"
      }
    }
  }
}
```

Creating product index

We will add four documents containing information about individual products using the following four corresponding requests:

Document 1

```
PUT /product/_create/1
{
  "name": "iPhone 12",
  "category": "electronics_mobile",
  "price": 999
}
```

Document 2

```
PUT /product/_create/2
{
  "name": "Samsung Galaxy S20",
  "category": "electronics_mobile",
  "price": 899
}
```

Document 3

```
PUT /product/_create/3
{
  "name": "Sony Bravia 4K TV",
  "category": "electronics_monitor",
  "price": 1499
}
```

Document 4

```
PUT /product/_create/4
{
  "name": "Dell XPS 13",
  "category": "electronics_laptop",
  "price": 1499
}
```

Query

Let's suppose we are interested in retrieving products belonging to the "electronics_mobile" category. To achieve this, we will utilize a term query that specifically targets the category field. By setting the search query to "electronics_mobile", we will ensure that only products with an exact match in the category field are returned.

We can use the following request to achieve this:

```
GET product/_search
{
    "query": {
      "term": {
        "category": "electronics_mobile"
      }
    }
}
```

Term query

Upon executing the term query with the search term "electronics_mobile", the query response will reveal the documents that exactly match the term. In this case, the documents with IDs 1 and 2 will be returned because they exactly match the term "electronics_mobile".

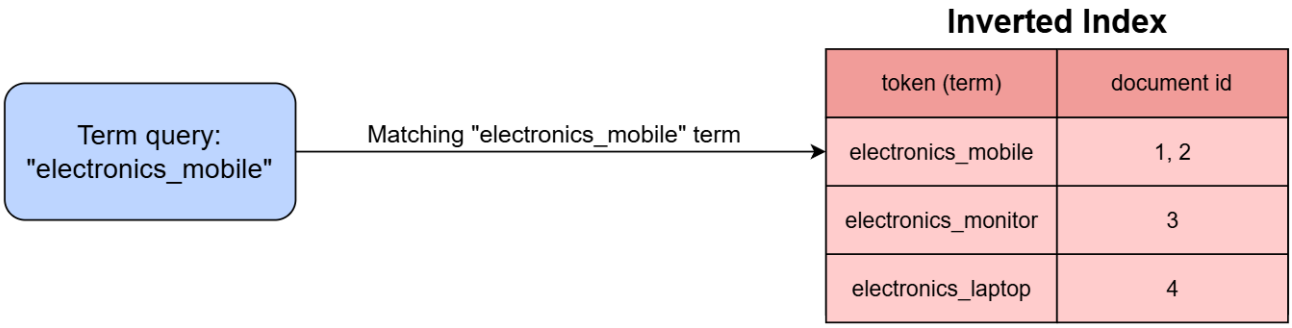
Explaining query result

Let's explore how the storage of the category field in the inverted index affects the query results. Because the category field is set as a keyword type, its values are stored in the inverted index without any modifications or analysis. Here is the inverted index representation of the category field after indexing the four documents.

Token (Term)	Document ID
electronics_mobile	1, 2
electronics_monitor	3
electronics_laptop	4

Inverted index

When executing a term query for "electronics_mobile", Elasticsearch does not apply any analysis to the search query. Instead, it directly passes the token (term) "electronics_mobile" to compare it with the values stored in the inverted index. As a result, the documents with IDs 1 and 2 are returned as they match the exact term "electronics_mobile".



Example of a term query flow

Term-Level Queries: Range Query

Range query

A **range query** is a type of query that allows us to retrieve data within a specified range of values. It is used to filter data based on a range of values, such as numbers, dates, or strings. For example, we can perform a range query to retrieve all documents where the price field falls within the range of 20 to 50.

Range queries are often used in conjunction with other types of queries, such as search queries and aggregation queries. For example, we can perform a search query to find all the documents that contain the word "apple" and then perform a range query to filter the results by price. This allows us to find all documents that contain the word "apple" and have a price between 20 and 50.

Range queries can be very efficient because they only need to scan data within the specified range. This makes them a good choice for large datasets.

Range query syntax

To perform a range query, we can send a GET request to the _search endpoint of the target index. In the request body, we define the query using the range query type and specify its field name. Within the field name, we can set the range condition.

We can define the condition using the following values:

- gt: Greater than
- gte: Greater than or equal to
- lt: Less than
- lte: Less than or equal to

Here's an example of the range query syntax for a range between 10 and 100:

```
GET my_index_name/_search
{
  "query": {
    "range": {
      "my_field_name": {
        "gt": 10,
        "lt": 100
      }
    }
  }
}
```

Range query syntax

Example

Let's explore a scenario where we have data stored in an Elasticsearch index, and our objective is to perform a range query to retrieve specific results. Subsequently, we will explain the process of the range query by breaking down the resulting query step by step.

Creating data

Let's create our product index, which contains the product data and specifies the following fields:

- name: It is stored as a keyword type.
- category: It is stored as a keyword type.
- price: It is stored as a float type.

The following request creates a product index and defines its field using explicit mappings:

```
PUT /product
{
  "mappings": {
    "properties": {
      "name": {
        "type": "text"
      },
      "category": {
        "type": "keyword"
      },
      "price": {
        "type": "float"
      }
    }
  }
}
```

Creating the product index

We will add six documents containing information about individual products using the following six corresponding requests:

Document 1

```
PUT /product/_create/1
{
  "name": "iPhone 12",
  "category": "electronics_mobile",
  "price": 1300
}
```

Document 2

```
PUT /product/_create/2
{
  "name": "Samsung Galaxy S20",
  "category": "electronics_mobile",
  "price": 1500
}
```

Document 3

```
PUT /product/_create/3
{
  "name": "Sony Bravia 4K TV",
  "category": "electronics_monitor",
  "price": 900
}
```

Document 4

```
PUT /product/_create/4
{
  "name": "Dell XPS 13",
  "category": "electronics_laptop",
  "price": 1350
}
```

Document 5

```
PUT /product/_create/5
{
  "name": "Samsung 85 Inch Smart Crystal UHD 4K",
  "category": "electronics_monitor",
  "price": 1200
}
```

Document 6

```
PUT /product/_create/6
{
  "name": "Dell LATITUDE 7000 13.3",
  "category": "electronics_laptop",
  "price": 1400
}
```

Query

Let's suppose we are interested in retrieving products with a price range between 1200 and 1500. To achieve this, we can use a range query By specifying the field as price and setting the conditions gte to 1200 and lte to 1500.

```
GET product/_search
{
  "query": {
    "range": {
      "price": {
        "gte": 1200,
        "lte": 1500
      }
    }
  }
}
```

Range query

The query above will retrieve documents with IDs 1, 2, 4, 5, and 6 because their prices fall within the range of 1200 and 1500.

Explaining the query result

Let's investigate how the storage of the price field in the inverted index impacts query results. As the price field is defined as a float type, its values are stored in the inverted index. However, in this case, instead of storing the field value as a separate term, the numerical value itself is inserted within the term.

Inverted Index

Token (Term)	Document ID
900	3
1200	5
1300	1
1350	4
1400	6
1500	2

The provided inverted index has certain limitations because Elasticsearch does not sequentially match each value within the specified range, such as 1200 to 1500, against the inverted index. This approach would be computationally intensive, especially for larger numbers. Instead, Elasticsearch optimizes

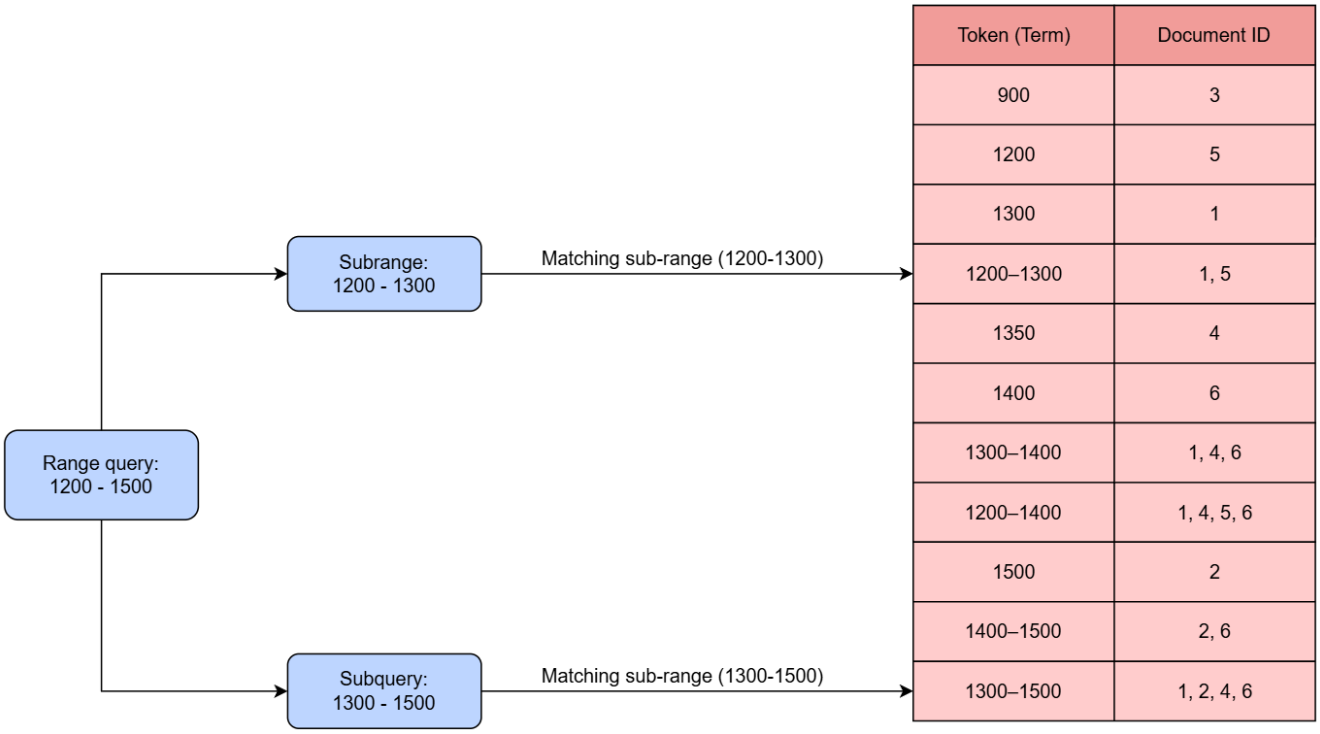
the search speed by automatically creating buckets (term ranges) for specific ranges and storing them as terms in the inverted index. In this case, Elasticsearch might create a term encompassing the range from 1200 to 1400, which includes the documents with the IDs 1, 4, 5, and 6. This approach significantly improves the efficiency of searching within ranges and reduces the computational overhead, especially when dealing with larger numbers.

Here is an illustrative example that demonstrates the structure of an inverted index with range terms. It is important to emphasize that this representation is provided for the purpose of comprehension and does not accurately depict the actual storage mechanism employed by Elasticsearch.

Token (Term)	Document ID
900	3
1200	5
1300	1
1200–1300	1, 5
1350	4
1400	6
1300–1400	1, 4, 6
1200–1400	1, 4, 5, 6
1500	2
1400–1500	2, 6
1300–1500	1, 2, 4, 6

Inverted Index with Buckets

In conclusion, when executing a range query in Elasticsearch, such as the range 1200 to 1500, the search process is optimized by splitting the range into buckets. This approach improves search speed and is well-suited for efficiently searching within the inverted index. By creating buckets that represent specific ranges, the query can be quickly matched against the indexed terms, making range queries faster and more suitable for searching within the inverted index. For example, instead of sequentially evaluating from 1200 to 1500, Elasticsearch can leverage term ranges such as (1200-1300) and (1300-1500) in the inverted index to efficiently retrieve the query results.



Range query flow of the illustrative example

Elasticsearch employs the concept of term ranges within the inverted index, not only for numeric values, but also for other types, such as dates. This indexing technique significantly enhances search efficiency, even with extensive datasets. For instance, consider searching through a million minutes of data. It transforms these minutes into a reduced number of minute ranges, typically just a few hundred, stored in the inverted index. Consequently, the search scope is dramatically reduced, leading to optimized performance.

Compound Queries: Boolean Query

Boolean query

A boolean query is a compound query type that enables the combination of multiple subqueries using boolean operators. These operators include the following:

- must: The must clause specifies criteria that must be satisfied by the matching documents. In other words, the documents must match the conditions specified in the must clause to be considered search results.
- must_not: The must_not clause excludes documents that match the specified conditions. It sets criteria that must not be satisfied by the documents for them to be considered valid matches.
- should: The should clause specifies the criteria that are desirable but not mandatory for the matching documents. Documents that satisfy the should conditions will have a higher relevance score, making them more likely to appear in the search results. However, documents that do not meet the should conditions can still be considered valid matches.
- filter: The filter clause is similar to the must clause in terms of mandatory conditions. However, unlike the must clause, the filter clause does not contribute to score calculation. It is primarily

used for filtering documents based on specific criteria, such as exact matches or range conditions, without affecting the relevance ranking.

By utilizing boolean queries, we can construct complex search conditions and control the scoring and relevance of the search results.

Boolean query syntax

To execute a boolean query, we can make a GET request to the _search endpoint of the designated index. In the request body, we define the query using the bool query type and specify the boolean operators. Each operator can contain an array of queries to match against the documents.

```
GET my_index_name/_search
{
  "query": {
    "bool": {
      "must": [ /* array of must queries */ ],
      "should": [ /* array of should queries */ ],
      "must_not": [ /* array of must_not queries */ ],
      "filter": [ /* array of filter queries */ ]
    }
  }
}
```

Boolean query syntax

Example

In this example, we will explore a scenario where we have data stored in an Elasticsearch index. We'll encounter a situation where we need to retrieve data based on multiple queries, and for each case, we will employ a boolean query to accomplish each task.

Creating data

Let's imagine we have an Elasticsearch index called products that contains information about various products available in an online store. Each document in the index represents a specific product and includes fields such as:

- name: It is stored as a text type.
- category: It is stored as a keyword type.
- price: It is stored as a float type.
- quantity: It is stored as an integer type.
- availability: It is stored as a keyword type.

The following request creates a product index and defines its field using explicit mappings:

```
PUT /product
{
  "mappings": {
    "properties": {
      "name": {
        "type": "text"
      },
      "category": {
        "type": "keyword"
      },
      "price": {
        "type": "float"
      },
      "quantity": {
        "type": "integer"
      },
      "availability": {
        "type": "keyword"
      }
```

Creating the product index

To add our data, we will add the following documents containing information about individual products using the following six corresponding requests:

Document 1

```
PUT /product/_create/1
{
  "name": "iPhone 12",
  "category": "Electronics",
  "price": 1099.99,
  "quantity": 50,
  "availability": "in stock"
}
```

Document 2

```
PUT /product/_create/2
{
  "name": "Samsung Galaxy S21",
  "category": "Electronics",
  "price": 899.99,
  "quantity": 25,
  "availability": "in stock"
}
```

Document 3

```
PUT /product/_create/3
{
  "name": "Sony PlayStation 5",
  "category": "Electronics",
  "price": 499.99,
  "quantity": 10,
  "availability": "out of stock"
}
```

Document 4

```
PUT /product/_create/4
{
  "name": "KitchenAid Stand Mixer",
  "category": "Home & Kitchen",
  "price": 349.99,
  "quantity": 15,
  "availability": "in stock"
}
```

Document 5

```
PUT /product/_create/5
{
  "name": "Nike Air Max 90",
  "category": "Fashion",
  "price": 129.99,
  "quantity": 30,
  "availability": "in stock"
}
```

Document 6

```
PUT /product/_create/6
{
  "name": "Amazon Echo Dot",
  "category": "Electronics",
  "price": 49.99,
  "quantity": 5,
  "availability": "in stock"
}
```

Queries

Let's consider the following query scenarios:

- Retrieving all the products in the "Electronics" category with a price less than 500.
- Retrieving all the products that are not in the "Electronics" category and are currently available ("in stock").
- Retrieving all the products with a price field between 100 and 1000, excluding those in the "Fashion" category, and prioritizing products that contain the term "Samsung".

Retrieving all the products in the "Electronics" category with a price less than 500

To execute this query, we can construct a boolean query with the following conditions in the must clause:

- A term query to match products with the category set to "Electronics".
- A range query to match products with prices lower than 500.

```
GET product/_search
{
  "query": {
    "bool": {
      "must": [
        {
          "term": {
            "category": "Electronics"
          }
        },
        {
          "range": {
            "price": {
              "lt": 500
            }
```

Bool query retrieving all the products in the Electronics category with a price less than 500

The query returns documents with IDs 3 and 6, which have the category "Electronics" and a price below 500.

Retrieving products that are not in the "Electronics" category and are currently available ("in stock")

To execute this query, we can use a boolean query with the following conditions:

- A must clause with a term query to match products whose availability is "in stock".
- A must_not clause with a term query to exclude products with the category set to "Electronics".

```
GET product/_search
{
  "query": {
    "bool": {
      "must": [
        {
          "term": {
            "availability": "in stock"
          }
        }
      ],
      "must_not": [
        {
          "term": {
            "category": "Electronics"
          }
        }
```

Bool query retrieving products that are not in the Electronics category and are currently in stock

The query retrieves documents with IDs 4 and 5, excluding those in the category of "Electronics" and only including those that are currently "in stock".

Retrieving products with the price field between 100 and 1000, excluding those in the "Fashion" category, and prioritizing products that contain the term "Samsung" in their name

To execute this query, we will utilize a boolean query with the following conditions:

- A must clause with a range query to match products with prices between 100 and 1000.
- A must_not clause with a term query to exclude products in the "Fashion" category.
- A should clause with a match query to prioritize products containing the term "Samsung" in the name field.

```
GET product/_search
{
  "query": {
    "bool": {
      "must": [
        {
          "range": {
            "price": {
              "gte": 100,
              "lte": 1000
            }
          }
        }
      ],
      "must_not": [
        {
          "term": {
            "category": "Fashion"
          }
        }
      ],
      "should": [
        {
          "match": {
            "name": "Samsung"
          }
        }
      ]
    }
```

Bool query retrieving products with the price range of 100–1000, excluding the Fashion category and prioritizing Samsung in the name

The query returns the documents with IDs 2, 3, and 4, satisfying the price range criteria and excluding those in the "Fashion" category. Among them, Document 2 is prioritized due to the presence of the term "Samsung" in its name.

Introduction to Aggregation

Aggregation in Elasticsearch is a powerful feature that allows us to perform data analysis and summarization on our indexed data. It enables us to gather, process, and present insights from our data in a structured and organized manner. Aggregations are similar to SQL's GROUP BY clause or data visualization's grouping and aggregation operations.

Aggregation can partition our data into meaningful subsets and compute various metrics on those subsets. These metrics can include things like counts, sums, averages, minimum and maximum values, statistical calculations, date histograms, and more. The result of an aggregation is a data structure that contains the computed metrics for each subset of data.

Applications of aggregation

Aggregation is a powerful tool in data analysis, with diverse applications across various industries. Here are some of them:

- In the context of e-commerce sales analysis, aggregations enable businesses to gain valuable insights into their performance. By utilizing aggregation, businesses can determine the top-selling product categories, average prices, and total revenue generated within specific time periods. With this information, businesses can identify trends, optimize marketing efforts, and efficiently manage their inventory.
- Aggregation plays a crucial role in social media analytics, where it can be applied to analyze user engagement metrics. This facilitates the identification of popular hashtags, user mentions, and trending topics based on the number of interactions, such as likes, comments, and shares. For social media marketers, these insights are invaluable because they can tailor their content strategy to enhance user engagement and reach a broader audience.
- Website owners can leverage Elasticsearch aggregation to study user behavior and engagement. By tracking metrics such as page views, unique visitors, and average session durations, businesses can gain a deeper understanding of their website's performance. Aggregating this data by time periods, regions, or referral sources allows them to identify peak traffic hours, popular content, and potential bottlenecks, enabling them to optimize the user experience and improve website performance.

Aggregation categories

Aggregation is a feature that allows us to summarize and analyze our data as metrics, statistics, or other valuable analytics. It enables us to derive meaningful insights from large datasets. In Elasticsearch, aggregations are organized into three main categories, each serving a specific purpose:

- Metric aggregation
- Bucket aggregation
- Pipeline aggregation

Metrics aggregations

Metrics aggregations in Elasticsearch are used to calculate various metrics or statistics on a numeric field in a collection of documents. They provide a way to perform data analysis and get insights into the numerical data stored in an Elasticsearch index. These aggregations help us answer questions like:

- What is the total sales revenue?
- What is the average age of customers?
- What is the maximum temperature recorded?
- What is the sum of all values?

There are several types of metrics aggregations available in Elasticsearch, each serving a different purpose:

- **Sum aggregation:** It calculates the total sum of the values in a numeric field.
- **Average aggregation:** It computes the average (mean) value of the numeric field.
- **Min or max aggregation:** It finds the minimum or maximum value in the numeric field.
- **Stats aggregation:** It provides multiple metrics, including count, sum, average, minimum, and maximum.
- **Extended stats aggregation:** It extends the stats aggregation with additional metrics such as standard deviation and sum of squares.
- **Value count aggregation:** It counts the number of values in the field.
- **Cardinality aggregation:** It estimates the number of unique values in the field.

Bucket aggregations

Bucket aggregations in Elasticsearch are a powerful method of grouping documents into buckets based on specific criteria. These aggregations are used to create data segments or categories, allowing us to perform analysis and gain insights into subsets of our data. Buckets can be created based on various factors such as existing field values, terms, customized filters, date ranges, numerical ranges, and more.

Bucket aggregations help us answer questions such as:

- How many products are there in each product category?
- How many times does each term appear in articles?
- How many sales occurred each month last year?

- How many customers fall into different age groups (e.g., 18–25, 26–35, 36–45)?
- What are the top n-selling products in a specific category?
- How many unique visitors accessed each website page during a specific time period?

There is a wide variety of bucket aggregations available in Elasticsearch, each differing in how they categorize documents into distinct buckets. The following are commonly used bucket aggregations:

- **Terms aggregation:** It groups documents based on unique terms found in a particular field.
- **Date histogram aggregation:** It groups documents into time-based intervals, enabling time-series analysis.
- **Range aggregation:** It divides documents into buckets based on predefined value ranges, facilitating analysis within specific ranges.
- **Histogram aggregation:** It groups documents into fixed-size numeric intervals for numeric data analysis.
- **Nested aggregation:** It creates nested buckets for documents with complex nested structures, allowing in-depth analysis of nested data.

Aggregation syntax

In Elasticsearch, we can perform aggregations alongside regular search queries using the _search API. This allows us to retrieve aggregated data and gain insights from our indexed documents.

To initiate a search request, we typically send a GET request to the index's _search endpoint. Within the request body, we can include the aggs property to specify the aggregations we want to perform. Each aggregation is defined by providing a unique aggregation_name and an associated aggregation_type.

Let's break down the key components:

- **size: 0:** This is an optional parameter that sets the number of search hits to return. When we set it to 0, Elasticsearch only returns the aggregated results, without any individual documents (query result).
- **query:** This section is also optional. We can define a query to filter the data before performing aggregations. If we want to aggregate all data, we can omit this section.
- **aggs:** This is the crucial part where we define the aggregations. Each aggregation is given a unique name (**e.g., aggregation_name**) and it consists of an **aggregation_type** (e.g., terms, sum, avg, max, min, etc.) with its specific options and parameters.

```
GET my_index_name/_search
{
    /* Optional: Setting size to 0 ensures
    the search results won't include individual documents.*/
  "size": 0,
  "query": {
    // Your query for filtering data (optional)
  },
  "aggs": {
    "aggregation_name": {
      "aggregation_type": {
        // Aggregation options and parameters
      }
    },
  }
}
```

Example

In this example, we have a limited product dataset. We will demonstrate the implementation of aggregation in Elasticsearch. We will begin with a basic aggregation request and then proceed to visualize the results using Kibana. These visualizations will prove beneficial whenever data analysis is necessary.

Creating data

To construct our products dataset, we can employ the following _bulk request, which will generate 50 distinct documents in the products index. Each document will represent a different product and encompass attributes such as name, description, price, category, brand, color, availability, rating, and seller.

Sample dataset: The _bulk request creating the products dataset

POST /products/_bulk

{ "index": { "_id": "1" } }

{ "name": "iPhone 13", "description": "The latest smartphone from Apple. Features a powerful A15 Bionic chip and a stunning Super Retina XDR display.", "price": 999, "category": "Electronics", "brand": "Apple", "color": "Midnight Blue", "availability": "In Stock", "rating": 4.5, "seller": "Apple Store" }

{ "index": { "_id": "2" } }

{ "name": "Samsung Galaxy S21", "description": "A high-end Android smartphone with a vibrant Dynamic AMOLED display and an impressive camera system.", "price": 899, "category": "Electronics", "brand": "Samsung", "color": "Phantom Black", "availability": "In Stock", "rating": 4.2, "seller": "Samsung Store" }

Aggregation request

For this aggregation request, we will employ the cardinality aggregation, which serves as a matrix aggregation to estimate the count of unique values within the specified field. Our focus will be on the brand field, where we intend to determine the number of distinct brands present in our dataset.

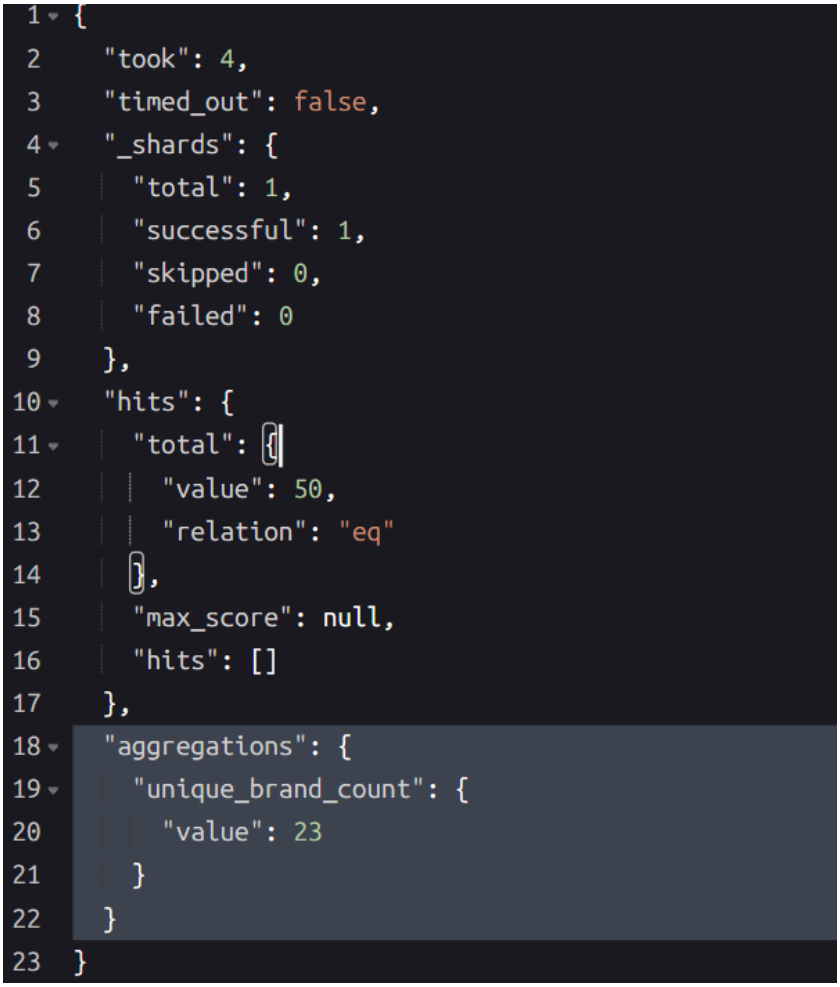
The following request will be directed to the _search endpoint of our products index, where we will set the size property to 0 because we only require the aggregation results and not the query results. Inside the request, under the aggs property, we will name the aggregation as unique_brand_count and define its type as cardinality with the field specified as "brand.keyword". When specifying "brand.keyword", we are being explicit about our intention to use the keyword field associated with the brand attribute. This is because Elasticsearch's dynamic mapping automatically generates two versions of each string field: one as text and the other as keyword.

```
1  POST /products/_search
2  {
3    "size": 0,
4    "aggs": {
5      "unique_brand_count": {
6        "cardinality": {
7          "field": "brand.keyword"
8        }
9      }
10   }
11 }
```

Aggregation request estimating the number of distinct brands

An aggregation request shares similarities with a search request in terms of providing search results. However, the key difference lies in the response format. While both requests return search results, the aggregation request offers an extra feature: the inclusion of an aggregations property in the response, which contains the result of the aggregation operation.

The response will include the properties took, time_out, shard, hits, and aggregations. The query property won't be returned because we specified size: 0 in the request. The aggregation result with the name unique_brand_count and a value of 23 can be found in the aggregations section in **lines 18–22**, indicating that there are 23 unique brands in our dataset.



Bucket Aggregation

Overview

Bucket aggregation is an aggregation type in Elasticsearch that categorizes documents into distinct groups or buckets based on specific criteria. Every bucket represents a value, serving to define the documents contained within it. By default, this representation corresponds to the document count within the bucket.

There is a wide variety of bucket aggregations available in Elasticsearch, each differing in how they categorize documents into distinct buckets. The following are commonly used bucket aggregations:

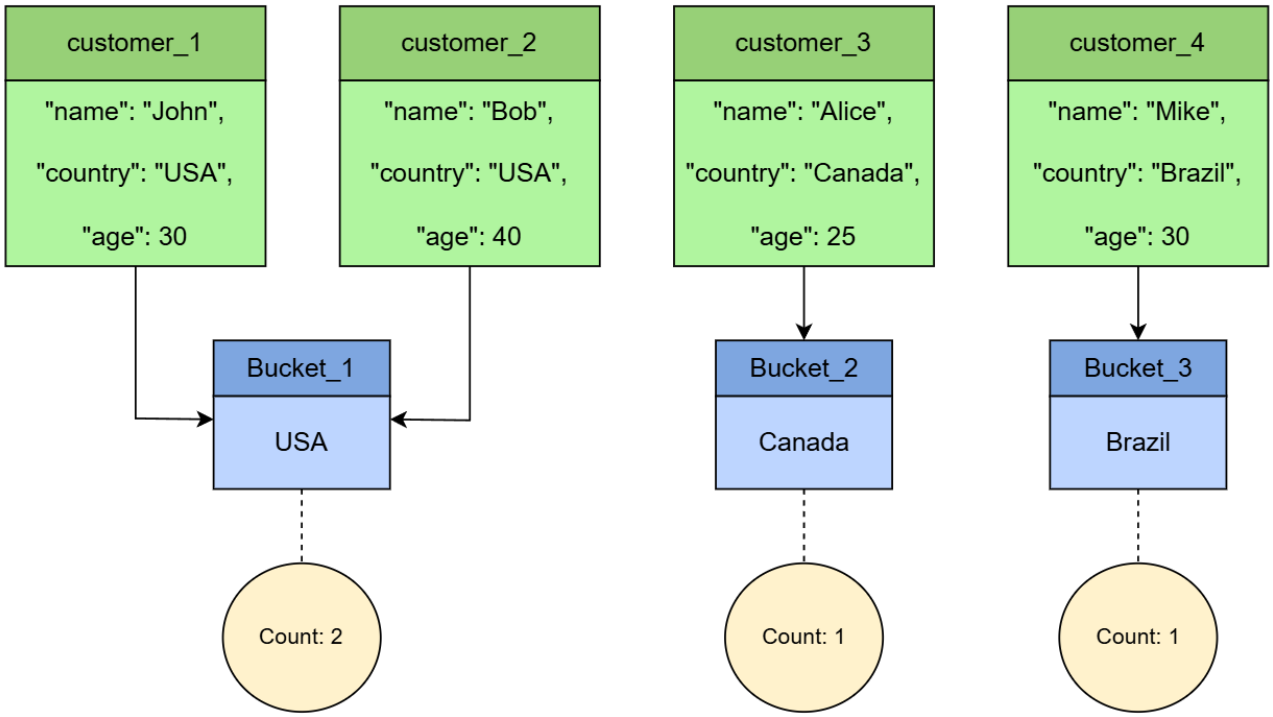
- Terms aggregation
- Range aggregation
- Histogram aggregation

Term aggregation

Term aggregation groups documents based on unique terms found within a specified field. It creates separate buckets for each distinct term in the chosen field and provides information about the documents associated with each term.

For example, if we have an index containing customer data and we want to apply a term aggregation on the “country” field, Elasticsearch will create a bucket for each unique country name present in that field. Each bucket will contain the relevant documents that match that particular country. The

following illustration visualizes an example of how the term aggregation on the “country” field would work.



Term aggregation on the country field

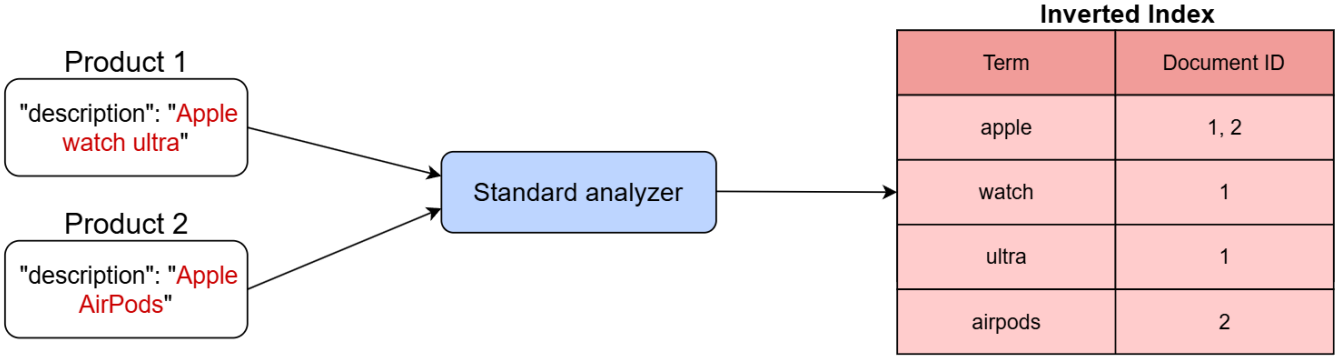
We can employ term aggregation on fields of types such as keyword, numeric, ip, boolean, or binary. Additionally, it is possible to utilize the text field. However, we need to enable the fielddata feature on the text field beforehand to generate buckets for the field’s analyzed terms.

During the application of term aggregation, a bucket is defined based on a term. In Elasticsearch, a term corresponds to what is stored within the inverted index. Consequently, when conducting term aggregation on a text field, a bucket is created for each term present in the inverted index, deriving from the index generated through analysis by the analyzer.

For example, let’s consider an Elasticsearch index containing product descriptions. The “description” field is of type “text” and is analyzed using the standard analyzer to break down the text into individual terms. Let’s suppose the index includes the following documents:

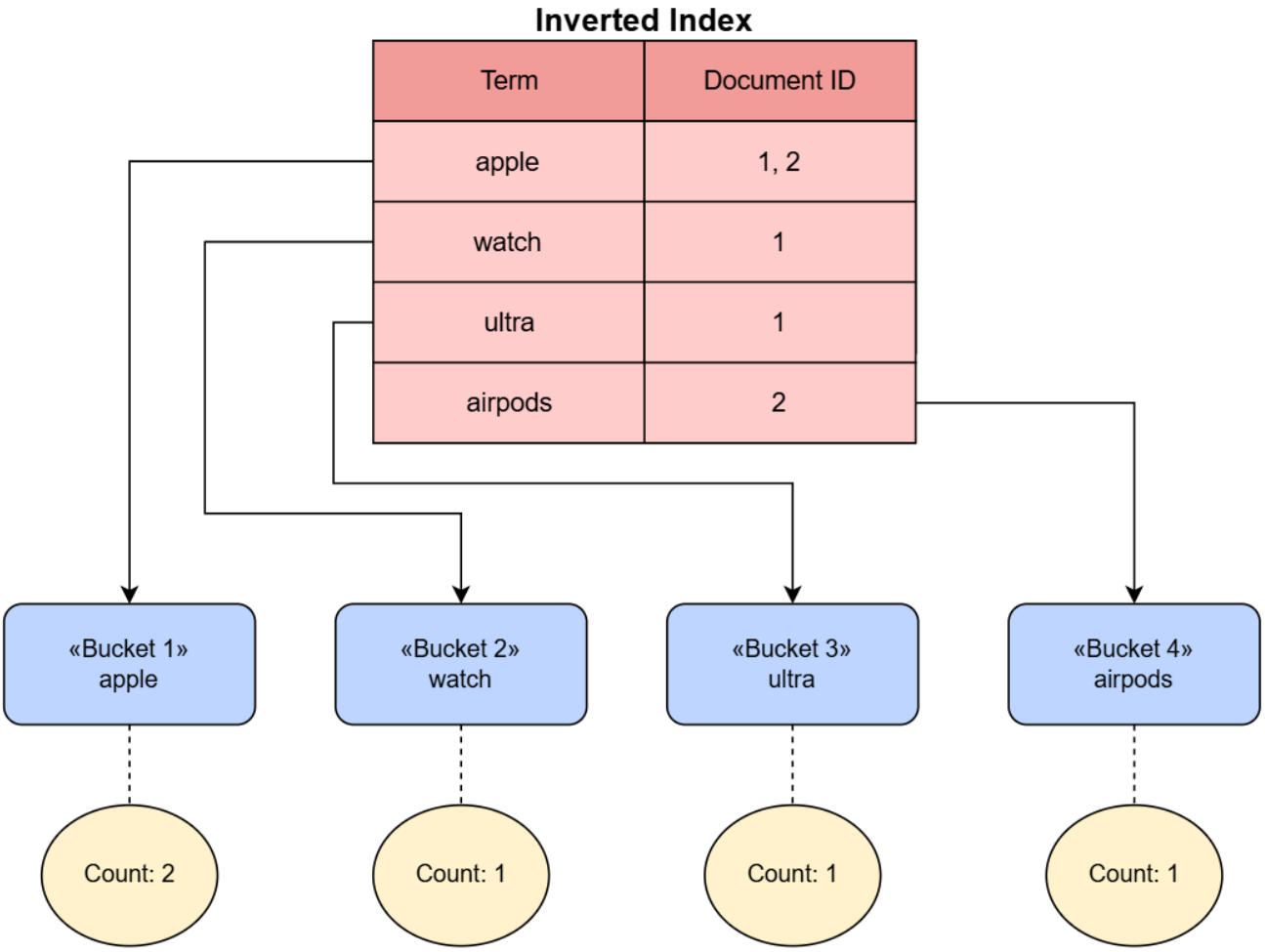
- Product 1 with the description,“Apple watch ultra”
- Product 2 with the description,“Apple AirPods”

The following image illustrates how the products are stored within the inverted index.



Inverted index for the description field

Upon implementing a term aggregation on the “description” field, Elasticsearch generates buckets based on the terms stored within the inverted index. These buckets include: “apple” (count: 2), “watch” (count: 1), “ultra” (count: 1), and “airpods” (count: 1).



Bucket aggregation on the description field

Range aggregation

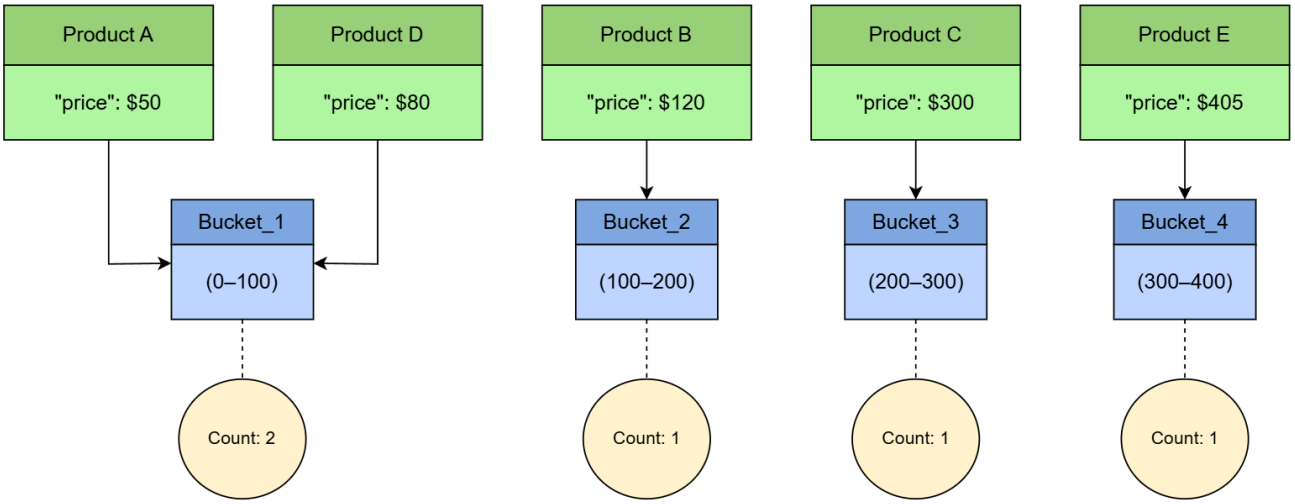
Range aggregation is a type of bucket aggregation that allows us to group documents into buckets based on specified ranges of values from a numeric or date field. This aggregation is particularly useful for analyzing and summarizing data within specific intervals or segments.

When using range aggregation, we have to define a set of ranges, each with specified “from” and “to” values. Documents with values falling within these ranges are grouped into corresponding buckets.

Let’s consider an example where we operate an e-commerce platform and seek to scrutinize the spread of product prices across various ranges. Our roster comprises product prices as follows:

- Product A: $50
- Product B: $120
- Product C: $300
- Product D: $80
- Product E: $405

By employing range aggregation, our objective is to establish categorizations that delve into product prices, delineating them within predefined intervals: (0$–$100), ($100–$200), ($200–$300), and ($300–$400).



Range aggregation on the price field

Histogram aggregation

Histogram aggregation is a type of bucket aggregation that helps us analyze numeric data by grouping it into specified intervals or bins. It allows us to create a histogram-like representation of our data, which can be useful for understanding the distribution and patterns within numeric fields.

Histogram aggregation is somewhat similar to range aggregation, but instead of specifying explicit ranges, we define the interval size, and Elasticsearch automatically creates ranges (bins) for us. For example, let's say we are analyzing the ages of customers in an online store. We want to understand how the customers are distributed across different age ranges. We have a list of customer ages as follows:

- Customer A: 18
- Customer B: 25
- Customer C: 28
- Customer D: 35
- Customer E: 41

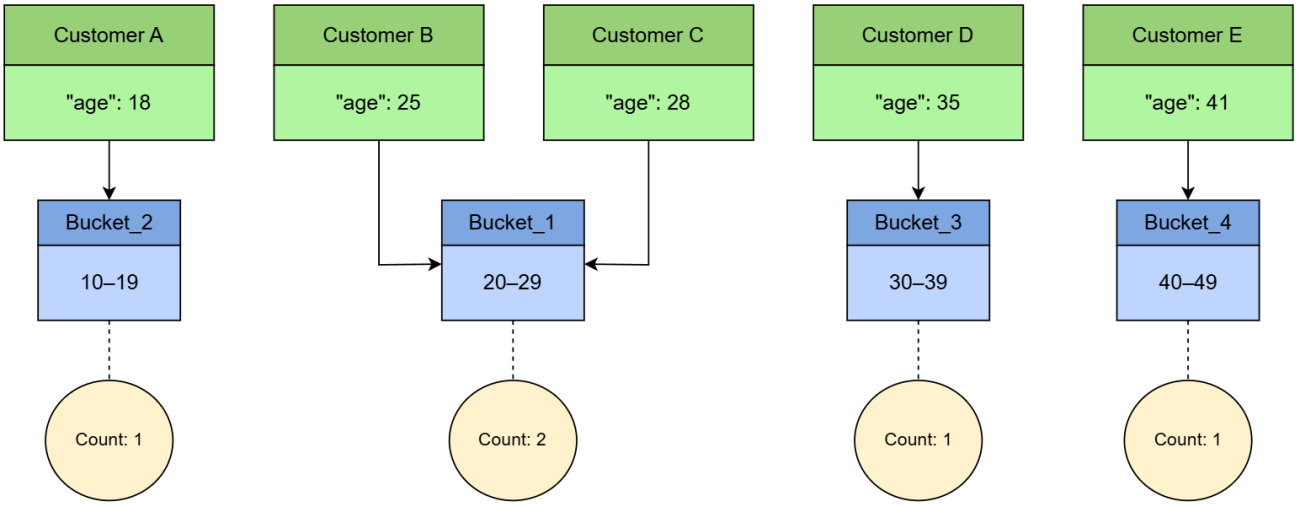
We don't define specific ranges using histogram aggregation as we do in range aggregation. Instead, we specify an interval size. Let's say we set the interval to 10 years.

Now, Elasticsearch will automatically create buckets (intervals) based on the interval size and the data's distribution. In this case, it might create the following buckets:

- 10–19 years
- 20–29 years
- 30–39 years
- 40–49 years

Elasticsearch also offers date range aggregation and date histogram aggregation. These aggregations function similarly to range and histogram aggregations, respectively, but are tailored for working with

date-based buckets. This specialization simplifies the process of constructing aggregations for date fields.



Histogram aggregation on the age field

Example

In this example, we have about 50 product entries. As we add these products, we'll be prompted to perform various aggregations on the dataset. After each aggregation request, we'll use Kibana to visualize the outcomes.

Creating data

First, we will create a product index that will contain the following fields:

- name: This text field stores the name of the product.
- description: This text field holds the description of the product, allowing for full-text search and analysis.
- price: It is a double type field that stores the price of the product as a floating-point number, enabling range queries and calculations.
- category: This keyword field captures the product's category as a single term. It's suitable for filtering and exact match queries.
- brand: A keyword field used to store the brand of the product. Being indexed as a keyword, it keeps the brand information intact and supports exact matches.
- color: It is another keyword field that maintains the color of the product as a single term.
- availability: This keyword field stores the availability status of the product, allowing exact match queries for availability-related analysis.
- rating: It is a float type field to hold the product's rating, facilitating numerical queries and calculations.
- seller: This keyword field contains the name of the seller and supports full-text search and analysis.

We can use the following request to create the product index with the described explicit mappings:

```
PUT product
{
  "mappings": {
    "properties": {
      "name": {
        "type": "text"
      },
      "description": {
        "type": "text"
      },
      "price": {
        "type": "double"
      },
      "category": {
        "type": "keyword"
      },
      "brand": {
        "type": "keyword"
      },
      "color": {
        "type": "keyword"
      },
      "availability": {
        "type": "keyword"
      },
      "rating": {
        "type": "float"
      },
      "seller": {
        "type": "keyword"
      }
    }
  }
}
```

Creating the product index

To construct our product dataset, we can employ the following _bulk request, which will generate 50 distinct documents in the product index:

Sample Dataset : The _bulk request creating the product dataset

POST /product/_bulk
{ "index": { "_id": "1" } }
{ "name": "iPhone 13", "description": "The latest smartphone from Apple. Features a powerful A15 Bionic chip and a stunning Super Retina XDR display.", "price": 999, "category": "Electronics", "brand": "Apple", "color": "Midnight Blue", "availability": "In Stock", "rating": 4.5, "seller": "Apple Store" }
{ "index": { "_id": "2" } }

{ "name": "Samsung Galaxy S21", "description": "A high-end Android smartphone with a vibrant Dynamic AMOLED display and an impressive camera system.", "price": 899, "category": "Electronics", "brand": "Samsung", "color": "Phantom Black", "availability": "In Stock", "rating": 4.2, "seller": "Samsung Store" }

Aggregation requests

Here are the aggregation requests we'll explore:

- Calculating the total product count for each seller.
- Grouping products into groups based on price, with each group representing a $500 interval.
- Grouping the "Electronics" category products by ratings (e.g., 0–0.99, 1–1.99, and 2–2.99) and determining the product count within each range.

Calculating the total product count for each seller

To form this aggregation, we can employ the term aggregation on the seller field. We will achieve this by sending a GET request to the _search endpoint of our index. Within the aggs section, we will designate the aggregation name as products_per_seller, and define the aggregation type as term. Within the term aggregation, we will specify the field as "seller".

```
GET product/_search
{
  "size": 0,
  "aggs": {
    "products_per_seller": {
      "terms": {
        "field": "seller"
      }
    }
  }
}
```

Term aggregation on the category field

Metric aggregation

Overview

Metrics aggregation is a type of aggregation that calculates metric data such as sum or average. It mainly refers to the mathematical calculations performed across a set of documents, usually based on the values of a numerical field present in the document. There are several types of metrics aggregations available in Elasticsearch, each serving a different purpose, such as:

- **Sum aggregation:** It calculates the sum of a numeric field across all documents matching the query.
- **Avg aggregation:** It computes the average of a numeric field across the matching documents.
- **Min aggregation:** It finds the minimum value of a numeric field among the documents.
- **Max aggregation:** It determines the maximum value of a numeric field among the documents.
- **Stats aggregation:** It provides a collection of basic statistics about a numeric field, including count, sum, average, min, and max.
- **Extended stats aggregation:** It is similar to the stats aggregation, but it includes additional statistical information such as standard deviation and variance.
- **Percentiles aggregation:** It computes one or more percentiles (e.g., 25th, 50th, and 75th) for a numeric field.
- **Cardinality aggregation:** It estimates the distinct count of values in a numeric or keyword field.
- **Value count aggregation:** It counts the number of non-null values in a numeric or keyword field.
- **The top_hits metric aggregator:** It keeps track of the most relevant document being aggregated. This aggregator is intended to be used as a sub-aggregator so that the top matching documents can be aggregated per bucket.

Metrics aggregation in Elasticsearch is a powerful tool for digging deep into our data and summarizing it. It helps us find important connections and patterns in our stored information. For example, we can use metrics aggregation to figure out which product is selling the most in a certain category. It's also helpful for getting statistics about products sold by a specific seller. By using it together with bucket aggregation, we can calculate metric values for different groups of data. This is called combined aggregation. This handy feature makes it much easier to understand different parts of our data and make smart decisions based on it.

Example

In this example, we have about 50 product entries. As we add these products, we'll be prompted to perform various aggregations on the dataset.

Creating data

First, we will create a product index that will contain the following fields:

- name: This text field stores the name of the product.
- description: This text field holds the description of the product, allowing for full-text search and analysis.
- price: It is a double type field that stores the price of the product as a floating-point number, enabling range queries and calculations.
- category: This keyword field captures the product's category as a single term suitable for filtering and exact match queries.

- brand: It is a keyword field used to store the brand of the product. Indexed as a keyword, it keeps the brand information intact and supports exact matches.
- color: It is another keyword field that maintains the color of the product as a single term.
- availability: This keyword field stores the availability status of the product, allowing exact match queries for availability-related analysis.
- rating: It is a float type field to hold the product's rating, facilitating numerical queries and calculations.
- seller: This keyword field contains the name of the seller and supports full-text search and analysis.

We can use the following request to create the product index with described explicit mappings:

Creating the product index

```
PUT product
{
 "mappings": {
  "properties": {
   "name": {
    "type": "text"
   },
   "description": {
    "type": "text"
   },
   "price": {
    "type": "double"
   },
   "category": {
    "type": "keyword"
   },
   "brand": {
    "type": "keyword"
   },
   "color": {
    "type": "keyword"
   },
   "availability": {
    "type": "keyword"
   },
   "rating": {
    "type": "float"
   },
   "seller": {
    "type": "keyword"
   }
  }
 }
}
```

To construct the product dataset, we can employ the following _bulk request, which will generate 50 distinct documents in the product index:

Sample Dataset: The _bulk request creating the product dataset

POST /product/_bulk
{ "index": { "_id": "1" } }
{ "name": "iPhone 13", "description": "The latest smartphone from Apple. Features a powerful A15 Bionic chip and a stunning Super Retina XDR display.", "price": 999, "category": "Electronics", "brand": "Apple", "color": "Midnight Blue", "availability": "In Stock", "rating": 4.5, "seller": "Apple Store" }
{ "index": { "_id": "2" } }
{ "name": "Samsung Galaxy S21", "description": "A high-end Android smartphone with a vibrant Dynamic AMOLED display and an impressive camera system.", "price": 899, "category": "Electronics", "brand": "Samsung", "color": "Phantom Black", "availability": "In Stock", "rating": 4.2, "seller": "Samsung Store" }

Aggregation requests

Here are the aggregation requests we'll explore:

- Aggregation to obtain statistical information about product prices.
- Determining the total sum of prices for products belonging to the "Apple" brand.
- Counting how many products include the word "pro" in their name.

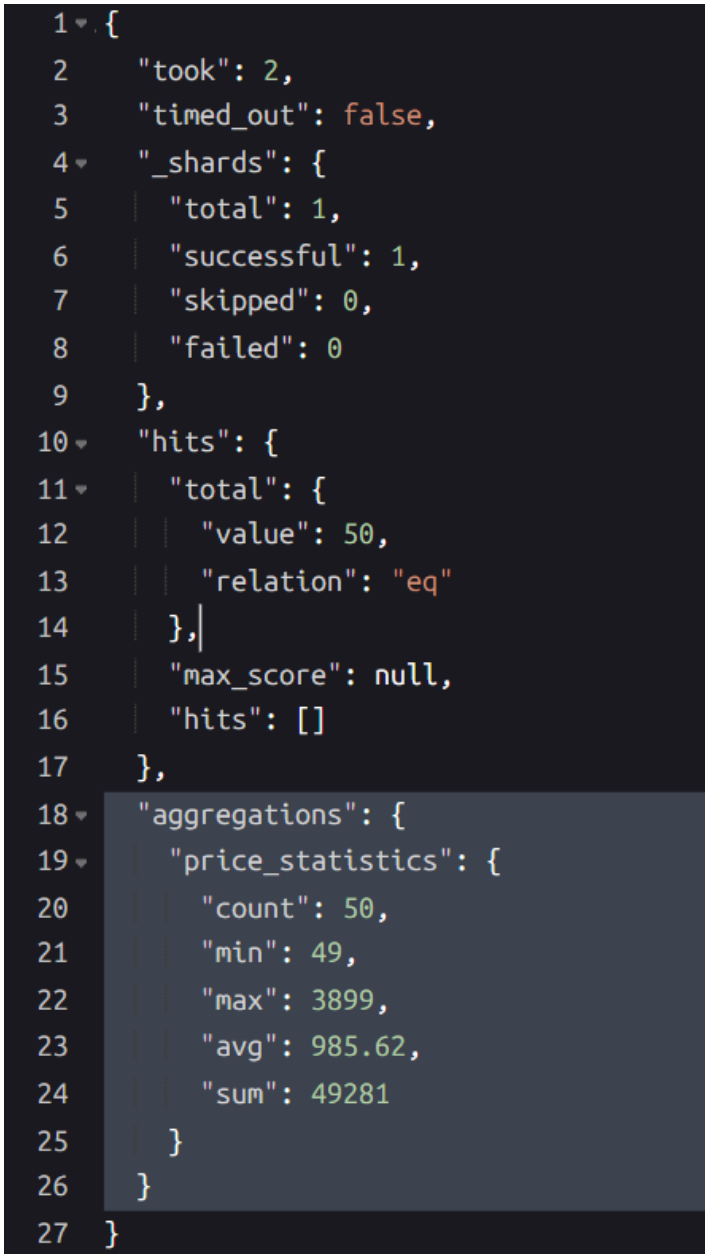
Aggregation to obtain statistical information about product prices

To use this aggregation, we'll use the stats aggregation on the price field. This can be done by sending a GET request to the _search endpoint of our index. In the aggs section, we will name the aggregation as price_statistics and choose the aggregation type as stats. Inside the stats aggregation, we will indicate the field as "price".

```
1  GET product/_search
2  {
3    "size": 0,
4    "aggs": {
5      "price_statistics": {
6        "stats": {
7          "field": "price"
8        }
9      }
10   }
11 }
```

Stats aggregation request on the price field

The following response includes an aggregation that computes statistical values for the price field. We can locate this aggregation on **lines 18–26**.



Response

Determining the total sum of prices for products belonging to the "Apple" brand

To apply this aggregation, we can utilize the sum aggregation on the price field. We will achieve this by sending a GET request to the _search endpoint of our index. Within the query section, we will add a term query to filter products belonging to the "Apple" brand. In the aggs section, we will add the aggregation name as apple_prices_sum and specify the aggregation type as sum. Inside the sum aggregation, we will indicate the field as "price".

```
GET /product/_search
{
  "size":0,
  "query": {
    "term": {
      "brand": "Apple"
    }
  },
  "aggs": {
    "apple_prices_sum": {
      "sum": {
        "field": "price"
      }
    }
  }
}
```

Sum aggregation price field with the term query to select the Apple brand

The provided response includes an aggregation that computes the sum of prices for products categorized under the "Apple" brand. This aggregation can be found within the range of **lines 18–22**.

```
{
  "took": 6,
  "timed_out": false,
  "_shards": {
    "total": 1,
    "successful": 1,
    "skipped": 0,
    "failed": 0
  },
  "hits": {
    "total": {
      "value": 5,
      "relation": "eq"
    },
    "max_score": null,
    "hits": []
  },
  "aggregations": {
    "apple_prices_sum": {
      "value": 4645
    }
  }
}
```

Response

Calculating the price percentiles for products whose names contain the word "pro"

To apply this aggregation, we can apply the percentile aggregation. This can be done by sending a GET request to the _search endpoint of our index. Within the query section, we will include a match query to filter products with names containing the term "pro". In the aggs section, we will name the aggregation as pro_name_percentiles and specify the aggregation type as percentiles. Inside the percentiles aggregation, we will designate the field as "price".

```
GET product/_search
{
  "size": 0,
  "query": {
    "match": {
      "name": "pro"
    }
  },
  "aggs": {
    "pro_name_percentiles": {
      "percentiles": {
        "field": "price"
      }
    }
  }
}
```

Percentiles aggregation with the match query to filter products with the pro term

The resulting response encompasses an aggregation that calculates percentiles for products with names containing the word "pro". We can locate this aggregation within the range of **lines 18–30**.


```
{
 "took": 4,
 "timed_out": false,
 "_shards": {
  "total": 1,
  "successful": 1,
  "skipped": 0,
  "failed": 0
 },
 "hits": {
  "total": {
   "value": 3,
   "relation": "eq"
  },
  "max_score": null,
  "hits": []
 },
 "aggregations": {
```

```
    "pro_name_percentiles": {
      "values": {
        "1.0": 248.99999999999997,
        "5.0": 249,
        "25.0": 436.5,
        "50.0": 999,
        "75.0": 2049,
        "95.0": 2399,
        "99.0": 2399
      }
    }
  }
}
```


Elasticsearch Use Cases: Log and Event Analytics, Search Applications, Monitoring and Observability.

Log and Event Analytics

Organizations generate vast amounts of log and event data from various sources, including applications, servers, network devices, and security systems. Analyzing this data is crucial for troubleshooting, performance monitoring, security analysis, and business intelligence.

Elasticsearch is widely used for log and event analytics due to its ability to ingest, store, search, and analyze this data in real-time.

How Elasticsearch is Used:

Elasticsearch serves as a central repository for log and event data. Tools like Logstash or Beats are commonly used to collect logs from diverse sources and then send them to Elasticsearch.

This centralization simplifies data management and enables comprehensive analysis across an entire infrastructure. Once data is in Elasticsearch, engineers can perform real-time searches and filtering, quickly sifting through millions of log entries to identify errors, warnings, or specific events. Its powerful query language allows for precise filtering based on time, severity, message content, and other relevant fields. Furthermore, Elasticsearch facilitates pattern recognition within log data, which is vital for detecting anomalies that might indicate potential system issues or security threats. For visualization and monitoring, Kibana, an integral part of the Elastic Stack, provides interactive dashboards that allow users to visualize log data, track trends, and monitor overall system health effectively.

Benefits:

Using Elasticsearch for log and event analytics offers several significant benefits. It enables faster troubleshooting by allowing engineers to quickly pinpoint the root cause of issues through efficient searching and correlation of logs across different systems. This leads to improved system performance as bottlenecks can be identified and resource utilization optimized. Security is also enhanced, as

analyzing security logs helps in detecting and responding to incidents more effectively by identifying suspicious activities. Ultimately, it contributes to greater operational efficiency by automating log analysis processes and significantly reducing manual effort.

Search Applications

Elasticsearch excels at powering search functionalities within various applications, from e-commerce websites to internal document management systems. Its full-text search capabilities, combined with its speed and scalability, make it an ideal choice for building robust search experiences.

How Elasticsearch is Used:

For website search, Elasticsearch provides fast and highly relevant results, enabling users to quickly find products, articles, or information. In e-commerce, it powers advanced features such as faceted search, autocomplete suggestions, and personalized recommendations, significantly enhancing the overall shopping experience. Beyond public-facing applications, Elasticsearch is also widely used in enterprise search solutions to index and search through vast repositories of documents, emails, and other forms of unstructured data. Its capabilities extend to geospatial search, allowing applications to search for data based on geographical locations, which is particularly useful for services like finding nearby restaurants or local businesses.

Benefits:

Implementing Elasticsearch for search applications leads to a significantly improved user experience by delivering fast, accurate, and highly relevant search results, which in turn increases user satisfaction. Features like autocomplete and personalized search keep users engaged and make the search process more intuitive. From an operational standpoint, Elasticsearch offers excellent scalability, capable of handling growing data volumes and increasing search traffic without compromising performance. Its flexibility is another key advantage, as it supports various data types and complex query requirements, making it adaptable to diverse search needs.

Monitoring and Observability

Monitoring and observability are critical for maintaining the health, performance, and availability of modern IT systems. Elasticsearch, as part of the Elastic Stack, provides a powerful platform for collecting, analyzing, and visualizing metrics, logs, and traces, offering a unified view of system behavior.

How Elasticsearch is Used:

Elasticsearch is central to collecting and storing various types of observability data, including metrics, logs, and application traces. Metrics, such as CPU usage, memory consumption, and network traffic,

are gathered from servers, containers, and applications using Beats or other data shippers and then indexed into Elasticsearch. Logs, as discussed previously, provide detailed records of events and system activities. Application traces, often collected using Elastic APM (Application Performance Monitoring), capture the end-to-end flow of requests through distributed systems, helping to identify latency and bottlenecks. All this data is then correlated within Elasticsearch, allowing for a holistic view of system performance and behavior. Kibana is used to create real-time dashboards that visualize these correlated metrics, logs, and traces, enabling engineers to monitor system health, identify anomalies, and diagnose issues proactively.

Benefits:

Utilizing Elasticsearch for monitoring and observability offers comprehensive insights into system operations. It enables proactive issue detection by continuously analyzing data for anomalies and deviations from normal behavior. This leads to faster root cause analysis, as engineers can quickly correlate different data types (logs, metrics, traces) to understand the underlying causes of performance degradation or failures. The unified view provided by the Elastic Stack improves operational efficiency, allowing teams to manage and respond to incidents more effectively. Ultimately, this contributes to higher system reliability and availability, ensuring that applications and services perform optimally and remain accessible to users.